**BÁO CÁO ĐỒ ÁN CÔNG NGHỆ THÔNG TIN**

**[CẦN ĐIỀN TÊN TRƯỜNG]**

**[CẦN ĐIỀN KHOA HOẶC VIỆN]**

# XÂY DỰNG HỆ THỐNG ĐẶT VÉ SỰ KIỆN ĐA HÌNH THỨC CÓ XỬ LÝ TRANH CHẤP ĐỒNG THỜI VÀ PHỤC HỒI GIAO DỊCH BẰNG KIẾN TRÚC HƯỚNG SỰ KIỆN

Sinh viên thực hiện: Lê Hữu Hào

Mã sinh viên: [CẦN ĐIỀN]

Lớp và khóa học: [CẦN ĐIỀN]

Ngành hoặc chuyên ngành: [CẦN ĐIỀN]

Giảng viên hướng dẫn: [CẦN ĐIỀN]

Địa điểm và thời gian nộp: [CẦN ĐIỀN]

# TÓM TẮT

Đồ án xây dựng hệ thống đặt vé sự kiện hỗ trợ hai hình thức: vé vào cửa không chỉ định chỗ, gọi là GA, và vé gắn với ghế định danh. Vấn đề trọng tâm là giữ đúng quyền sở hữu tài nguyên khi nhiều yêu cầu cùng đến, đồng thời phục hồi kết quả nghiệp vụ khi broker hoặc bộ xử lý nền tạm ngừng. Hệ thống gồm API và các worker Spring Boot trong một modular monolith, giao diện React, PostgreSQL, Flyway và Kafka. Thanh toán và hoàn tiền đều là mô phỏng, không có giao dịch tiền thật.

Giải pháp sử dụng cập nhật tồn kho có điều kiện cho GA, khóa ghế theo thứ tự ổn định và idempotency trong cùng giao dịch PostgreSQL. Expiry Worker giải phóng giữ chỗ mà không cần có yêu cầu đặt mới. Kết quả thanh toán được ghi bền bằng đồng hồ cơ sở dữ liệu trước khi xử lý; kết quả thành công sau khi booking mất quyền giữ chỗ dẫn đến REFUND_REQUIRED và nghĩa vụ bù trừ, không phát vé hoặc nhận lại tài nguyên.

Transactional outbox ghi sự kiện cùng giao dịch nghiệp vụ. Publisher gửi sau commit; consumer ghi inbox cùng hiệu ứng xử lý trước khi xác nhận offset. Retry hữu hạn, dữ liệu lỗi bền, DLQ và replay có phân quyền hỗ trợ phục hồi. Thiết kế chấp nhận gửi ít nhất một lần và chống hiệu ứng lặp; không cam kết exactly-once xuyên PostgreSQL và Kafka.

Gate T9.1 và T9.3 đã chạy 130 test backend không bỏ qua test. Phép đo T9.2 gồm ba burst 50 request cho mỗi hình thức vé, tổng 300 request; cả sáu burst có oversell bằng 0 và không có lỗi ngoài conflict dự kiến. Kịch bản T10.2 đạt tám bước trong 365,499 giây, gồm 310 giây xem kết quả và thuyết minh định giờ; launcher chỉ chạy 9 test và bỏ qua 121 integration test. Các con số này có phạm vi khác nhau, không được cộng hoặc thay thế cho nhau. [E1–E3, E6]

Kết quả hỗ trợ kết luận rằng các bất biến được bảo toàn trong những ca thử đã chạy. Chưa có bằng chứng về tải duy trì, nhiều broker, mất ổ đĩa, mọi cửa sổ crash hoặc vận hành thương mại. Báo cáo trình bày cả kết quả và giới hạn này để người đọc có thể kiểm tra, tái hiện và đánh giá đúng phạm vi của sản phẩm.

Từ khóa: đặt vé, tranh chấp đồng thời, PostgreSQL, idempotency, transactional outbox, Kafka, phục hồi giao dịch.

# MỤC LỤC

**CHƯƠNG 1 BÀI TOÁN VÀ MỤC TIÊU** ..... 6

1.1 Bài toán đặt vé đa hình thức ..... 6

1.2 Mục tiêu phạm vi và đối chiếu kế hoạch ..... 7

**CHƯƠNG 2 YÊU CẦU VÀ BẤT BIẾN** ..... 8

2.1 Tác nhân và chức năng ..... 8

2.2 Yêu cầu phi chức năng và bảo mật ..... 9

2.3 Các bất biến dùng để nghiệm thu ..... 10

**CHƯƠNG 3 KIẾN TRÚC VÀ CÁC QUYẾT ĐỊNH** ..... 11

3.1 Kiến trúc tổng thể ..... 11

3.2 Stack và phân chia module ..... 12

3.3 Ranh giới giao dịch và thứ tự khóa ..... 13

3.4 Dữ liệu bền và truyền sự kiện ..... 14

3.5 Các lựa chọn và giới hạn kiến trúc ..... 15

**CHƯƠNG 4 MÔ HÌNH DỮ LIỆU VÀ TRẠNG THÁI** ..... 16

4.1 Quan hệ dữ liệu lõi ..... 16

4.2 Catalog tồn kho và idempotency ..... 17

4.3 Ledger vé outbox và dữ liệu phục hồi ..... 18

4.4 State machine của booking và tài nguyên ..... 19

4.5 Migration và dữ liệu mẫu tái lập ..... 20

**CHƯƠNG 5 TRIỂN KHAI CÁC LUỒNG NGHIỆP VỤ** ..... 21

5.1 Giữ và bán vé GA ..... 21

5.2 Ghế định danh và đặt nhiều ghế nguyên tử ..... 22

5.3 Idempotency cho booking và payment ..... 23

# MỤC LỤC TIẾP THEO

5.4 Expiry Worker và nhả hold....24
5.5 Thanh toán mô phỏng và phân xử với expiry....25
5.6 Phát vé và bù trừ mô phỏng....26
5.7 Outbox publisher sau commit....27
5.8 Consumer retry DLQ và dữ liệu lỗi bền....28
5.9 Replay có quyền và audit....29
5.10 Giao diện logging metrics và health....30
**CHƯƠNG 6 PHƯƠNG PHÁP KIỂM THỬ....31**
6.1 Các lớp kiểm tra và môi trường....31
6.2 Thiết kế phép đo 50 request đồng thời....32
**CHƯƠNG 7 KẾT QUẢ THỰC NGHIỆM....33**
7.1 Các gate và cách đọc số test....33
7.2 Số liệu tải theo từng lượt....34
7.3 Đối chiếu PostgreSQL sau hold và sau bán....35
7.4 Kafka outage duplicate retry DLQ và replay....36
7.5 Payment expiry rollback và khởi động lại....37
7.6 Demo sạch và lượt tập 5 đến 10 phút....38
**CHƯƠNG 8 ĐÁNH GIÁ VÀ KẾT LUẬN....39**
8.1 Kết quả đạt được và hướng phát triển....39
**TÀI LIỆU THAM KHẢO....40**
**PHỤ LỤC A LỆNH TÁI HIỆN TRÊN WINDOWS....41**
**PHỤ LỤC B TRUY VẤN ĐỐI CHIẾU CHỈ ĐỌC....42**
**PHỤ LỤC C MA TRẬN TRUY VẾT VÀ BÀN GIAO....43**

# THUẬT NGỮ VÀ CÁCH ĐỌC BẰNG CHỨNG

| Thuật ngữ | Ý nghĩa trong đồ án |
|---|---|
| GA | General Admission, vé theo số lượng, không có ghế định danh |
| Hold | Quyền giữ vé hoặc ghế tạm thời, chưa phải vé đã bán |
| Booking | Đơn giữ chỗ có vòng đời và lịch sử trạng thái |
| EDA | Kiến trúc dùng sự kiện để phân phối thay đổi đã ghi nhận |
| Outbox và inbox | Sổ sự kiện cần gửi và biên nhận chống xử lý lặp |
| DLQ | Dead Letter Queue, topic thông báo bản ghi không xử lý được |
| Replay | Gửi lại sự kiện gốc có audit và giới hạn, giữ eventId |
| TTL | Thời gian giữ chỗ; cấu hình thường 600 giây |
| p50 p95 p99 | Phân vị độ trễ của các mẫu đo thực tế |
| No-op | Đã nhận yêu cầu nhưng không tạo hiệu ứng nghiệp vụ mới |

Các mã [E1] đến [E6] trỏ đến bằng chứng nội bộ liệt kê ở Tài liệu tham khảo. [D1] là bộ đặc tả hiện hành và ADR; [S1] là mã nguồn, migration và hợp đồng trong repository. Một giá trị trong bảng kết quả phải được đọc cùng tên gate, dữ liệu thử và trường JSON tương ứng. PASS chỉ áp dụng cho những kiểm tra đã ghi nhận, không có nghĩa hệ thống đúng trong mọi điều kiện.

Snapshot trước và sau là ảnh dữ liệu truy vấn PostgreSQL, không phải ảnh chụp màn hình. Các sơ đồ trong báo cáo do tác giả dựng từ đặc tả và mã nguồn, không phải ảnh Chrome. Xác nhận Chrome T10.1 có nguồn là lời xác nhận của chủ dự án; không có file ảnh được agent truy cập. Kiểm tra DOM, jsdom với HTTP thật và kiểm tra trực quan Chrome là các loại bằng chứng khác nhau.

Mốc đối chiếu mã nguồn là commit 984e95d7dc7aca0b7ea94743e27e21d5d713d906 cùng các thay đổi tài liệu và harness T10.1–T10.2 chưa commit. T9.1 và T9.2 có commit gốc khác được ghi trong evidence; số đo cũ không được tự gắn cho phiên bản mới như một lượt chạy mới. Kế hoạch Word ngày 12/11/2026 chỉ mô tả mục tiêu và tình trạng tại thời điểm đó.

# CHƯƠNG 1 BÀI TOÁN VÀ MỤC TIÊU

## 1.1 Bài toán đặt vé đa hình thức

Một sự kiện có thể bán quyền vào cửa theo số lượng hoặc gắn vé với vị trí cụ thể. Với GA, khách chọn số vé trong một hạng giá; hệ thống chỉ cần biết tổng số còn bán được. Với ghế, từng tài nguyên có định danh, nhãn và người đang giữ. Hai hình thức có giao diện khác nhau nhưng đều cần một vòng đời booking thống nhất và một quy tắc phân xử thanh toán đáng tin cậy.

Tranh chấp xuất hiện khi nhiều request cùng đọc một snapshot availability rồi cùng đặt. Nếu mỗi request đọc số còn lại và cập nhật không có điều kiện trong những giao dịch độc lập, các lần kiểm tra có thể cùng hợp lệ trong khi tổng kết quả vượt capacity. Với ghế, đọc trạng thái AVAILABLE không tự cấp quyền sở hữu; chỉ giao dịch ghi đã commit mới xác lập được quyền đó.

Giữ chỗ còn làm bài toán phức tạp hơn. Tài nguyên có thể hết hạn trong lúc payment đang được ghi nhận, consumer chưa xử lý hoặc người khác đang chờ đặt. Việc frontend tự đổi trạng thái, yêu cầu mới tự dọn hold hoặc consumer tin thời gian do client gửi đều có thể gây nhả nhầm tài nguyên hoặc phát vé cho booking đã mất quyền giữ chỗ.

Một vấn đề khác là hai hệ lưu trữ không có giao dịch nguyên tử chung trong triển khai này. Nếu cập nhật booking rồi gửi Kafka trực tiếp, tiến trình có thể dừng giữa hai thao tác. Kết quả nghiệp vụ đã commit nhưng sự kiện chưa gửi, hoặc sự kiện đã gửi nhưng bên nhận chưa biết lần xử lý trước đã thành công. Đồ án giải quyết bằng dữ liệu bền, xử lý lặp an toàn và các điểm phục hồi có kiểm soát, thay vì giả định mạng và broker luôn hoạt động.

Đối tượng phục vụ gồm khách demo, quản trị viên và người đánh giá kỹ thuật. Sản phẩm cần đủ giao diện để quan sát một luồng xuyên suốt, nhưng trọng tâm nghiên cứu là toàn vẹn giao dịch, khả năng tái hiện tranh chấp và bằng chứng phục hồi. Phạm vi này phù hợp với đồ án cá nhân định hướng backend; không đặt mục tiêu vận hành bán vé thương mại. [D1: PRD §§1–4]

## 1.2 Mục tiêu phạm vi và đối chiếu kế hoạch

Mục tiêu chức năng là cho khách xem hai loại sự kiện, giữ tài nguyên, thanh toán mô phỏng, xem vé và lịch sử. Mục tiêu kỹ thuật là không bán vượt sức chứa, không giữ trùng ghế, không tạo booking lần hai khi gửi lại cùng lệnh, không nhả hoặc phát vé lặp và không mất kết quả đã commit khi Kafka tạm ngừng.

| Mục tiêu kế hoạch cũ | Hiện trạng được đối chiếu |
|---|---|
| Khung Java React PostgreSQL | Đã có source, cấu hình và migration V1–V9 |
| Luồng GA và ghế | Có API, giao diện, test PostgreSQL và phép đo riêng |
| Expiry và payment muộn | Có worker và processor chung; late success đi bù trừ |
| Kafka và phục hồi | Có outbox, inbox, retry, failure DB, DLQ, replay audit |
| Demo và báo cáo | T10.1–T10.2 có bằng chứng; T10.3 là báo cáo này |

File Quy_Trinh_Xay_Dung_He_Thong_Dat_Ve_Le_Huu_Hao.docx từng ghi nhiều phần chưa triển khai. Nhận định đó phản ánh kế hoạch ngày 12/11/2026, không mô tả repository ở mốc đối chiếu hiện tại. Báo cáo giữ mục tiêu từ kế hoạch, lấy chức năng và kết quả từ mã nguồn, PROGRESS và evidence đã ghi nhận; không dùng tình trạng cũ để phủ định hoặc chứng nhận kết quả mới.

Trong MVP không có thu tiền, hoàn tiền qua ngân hàng, dữ liệu thẻ, QR có chữ ký dùng ngoài mạng, check-in production, định giá động hay nhiều nhà tổ chức. Hệ thống cũng không triển khai Redis, Kubernetes hoặc các microservice riêng. Những giới hạn này là lựa chọn phạm vi, không phải các tính năng đã kiểm chứng nhưng bị ẩn khỏi demo.

Tiêu chí đánh giá ưu tiên chứng cứ có thể kiểm tra: response API khớp booking đã commit, SQL phản ánh allocation, lịch sử không lặp, sự kiện được giữ khi lỗi và replay không tạo hiệu ứng lần hai. Kết quả giao diện phải lấy từ API thật. Một ảnh đẹp hoặc một commit thành công không thay được các tiêu chí này. Các mốc task được dùng để truy vết công việc, không quy đổi thành phần trăm chất lượng sản phẩm. [D1; E1–E6]

# CHƯƠNG 2 YÊU CẦU VÀ BẤT BIẾN

## 2.1 Tác nhân và chức năng

Khách demo được phép tạo booking, xem booking của mình, xem lịch sử, chạy mô phỏng payment và xem vé. ADMIN quản lý catalog, xem snapshot tồn kho, lọc booking và điều tra hoặc replay sự kiện. Persona trên giao diện chỉ chọn bộ thông tin xác thực fixture; quyền truy cập được quyết định ở server.

| Nhóm API | Quyền và kết quả chính |
|---|---|
| GET events và availability | Public; danh sách, chi tiết, snapshot GA hoặc ghế |
| POST bookings | CUSTOMER; key bắt buộc, trả202 với bookingId và expiresAt |
| GET booking và tickets | Owner hoặc ADMIN; không lộ booking khách khác |
| GET me/bookings | Lịch sử của principal đã xác thực |
| POST simulate-payment và cancel | CUSTOMER owner, môi trường local/test theo policy |
| Admin catalog và inventory | ADMIN; chặn thay đổi phá capacity hoặc allocation |
| Admin outbox và recovery | ADMIN local/test; metadata, audit và replay hữu hạn |

Hợp đồng OpenAPI 3.1 có 18 operation ở mốc hiện hành; bộ kiểm tra hợp đồng gồm 69 trường hợp dương và âm. Đây là số operation nghiệp vụ, không cộng các endpoint Actuator. Hợp đồng định nghĩa shape request, response, quyền và lỗi; kiểm tra schema không tự chứng minh transaction runtime đúng. Các gate HTTP và SQL bổ sung lớp kiểm chứng đó. [S1: docs/contracts/openapi.json; E1.results.contract; E6.checks.contracts]

Một booking chọn một sự kiện và một loại vé. GA dùng quantity từ 1 đến 6; SEATED dùng danh sách từ 1 đến 6 UUID ghế khác nhau. Server tính tổng tiền theo giá đang khóa và lưu snapshot vào item; client không khai userId, role, giá bán hoặc timestamp payment để quyết định nghiệp vụ.

Admin có thể thay đổi giá cho đơn tương lai và capacity GA trong giới hạn held + sold. Hệ thống không mở đường xóa ghế đã giữ/bán hoặc thay mode/danh sách ghế bằng PATCH. Với API tạo catalog chưa có idempotency, giao diện không tự gửi lại khi mất phản hồi; người dùng cần đọc danh sách để xác định kết quả trước khi tạo tiếp. [D1; ADR0003; ADR0008]

## 2.2 Yêu cầu phi chức năng và bảo mật

Tính đúng đắn được đặt trước throughput. Mọi thay đổi phân bổ phải nằm trong giao dịch PostgreSQL có kiểm tra trạng thái và quyền sở hữu. Lỗi khóa, statement, pool hoặc deadline không được trả thành công trước khi giao dịch đã commit hoặc rollback. Kết quả không rõ sau mất mạng phải được tra cứu bằng idempotency, không tạo thao tác mới để đoán lại.

| Chính sách khởi đầu | Giá trị và ý nghĩa |
|---|---|
| Hold TTL và expiry | 600 giây; worker5 giây, batch50 booking |
| Khóa và statement | 1500ms và5000ms trong transaction |
| Attempt và tổng nghiệp vụ | 5000ms và10000ms; không cấp lại deadline cho mỗi retry |
| Pool và request | Chờ connection2000ms; ngân sách HTTP15000ms |
| Body command và catalog | 8192 byte và131072 byte UTF-8 |
| CORS local | localhost hoặc127.0.0.1, đúng cổng5173; không credentials |

Các giá trị trên là cấu hình baseline, không phải SLA đã đo. Request JSON phải có cấu trúc hợp lệ, giới hạn độ sâu và không có field lạ hoặc key lặp. Lỗi công khai dùng mã và correlationId, không trả SQL, stack trace, raw header hoặc payload thanh toán. Sai token trả401; CUSTOMER gọi admin trả403; việc đọc booking của khách khác được che bằng404 theo hợp đồng.

Mock bearer chỉ dùng local/test và ánh xạ server sang tài khoản seed. Bật mock auth cùng profile ngoài local/test làm startup thất bại. Không nhận X-User-Id hoặc role tự khai. Bộ chọn persona, ẩn nút và CORS không thay thế kiểm tra authorization tại API/service. Token không lưu trong database hoặc browser storage của command; storage chỉ giữ thông tin cần retry ổn định.

Khả năng quan sát gồm log JSON có correlationId, bookingId và message eventId, health và metrics có label hữu hạn. Bảo mật log không chỉ là che password trong một dòng; phải tránh dump cấu hình, query thật, token và body. Kiểm tra secret chỉ chứng minh phạm vi đã quét tại mốc đo, không bảo đảm mọi cấu hình DEBUG tương lai an toàn. [ADR0002; ADR0008; S1: shared, identity, operations]

## 2.3 Các bất biến dùng để nghiệm thu

Với mỗi loại GA, capacity, held và sold phải không âm; held + sold không vượt capacity. Bộ đếm được đối chiếu với tổng quantity của booking còn PENDING_PAYMENT hoặc đã CONFIRMED, không chỉ nhìn số request trả202. Với SEATED, mỗi ghế có tối đa một owner hiện hành; booking_items lưu lịch sử chọn ghế nên không có unique seat_id xuyên mọi booking lịch sử.

| Bất biến | Cách kiểm tra |
|---|---|
| Không oversell GA | held+sold≤capacity và khớp tổng quantity đang phân bổ |
| Không giữ trùng ghế | Đếm allocation active theo seatId; owner/state phải khớp |
| Không booking dở | Booking thành công có item, key hoàn chỉnh, history và outbox |
| Không phát vé sai | Chỉ CONFIRMED có vé; số vé/ordinal đúng quantity và ghế |
| Không ghi nghiệp vụ lặp | Một transition/cause hiệu lực; inbox riêng từng consumer |
| Không hoàn tồn hai lần | Terminal/no-op không giảm held hay đổi owner mới |
| Không bù trừ trùng | Một nghĩa vụ/payment; COMPLETED không chạy lại |
| Không mất sự kiện bền | Outbox chưa publish vẫn giữ record; lỗi cuối có audit |

Một hàng GA có CHECK bảo vệ giới hạn số lượng. Các điều kiện xuyên bảng như số vé bằng tổng item hoặc owner còn hợp lệ cần domain transaction và truy vấn đối chiếu; không thể thay tất cả bằng một CHECK. Tương tự, unique vé ngăn trùng định danh nhưng không tự chứng minh vé thuộc booking hợp lệ.

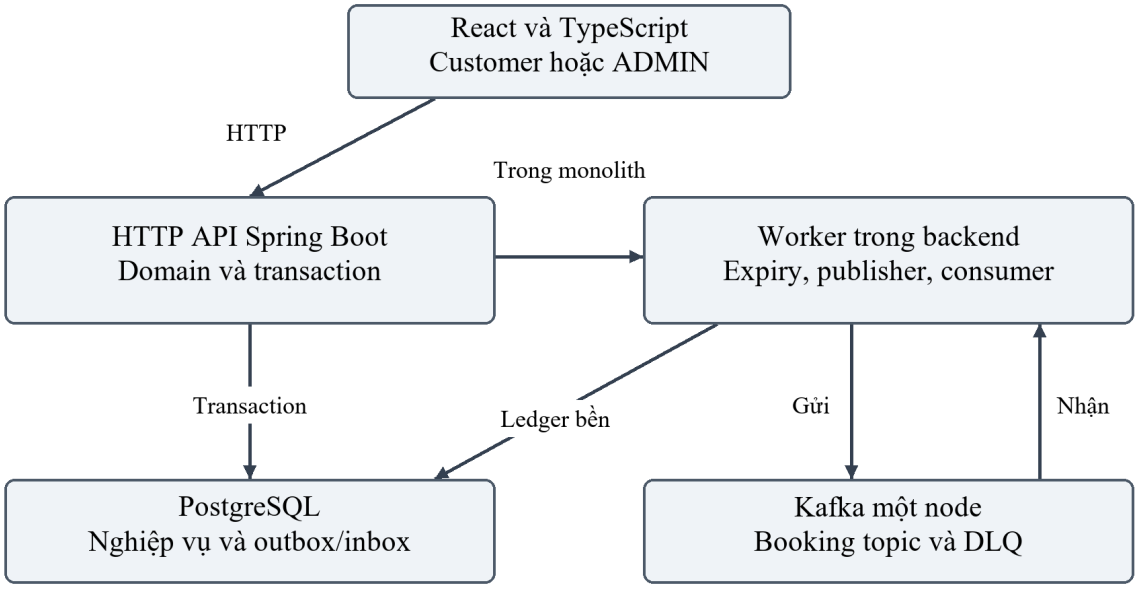
Định nghĩa “đúng một lần” trong yêu cầu nhả hold là hiệu ứng nghiệp vụ quan sát được, không phải cam kết giao nhận message đúng một lần. Worker có thể chạy lặp, publisher gửi lại, consumer restart và request HTTP retry. Tất cả phải kiểm tra dữ liệu bền trước khi chuyển trạng thái. Tính chất này được kiểm bằng fault injection và replay, không suy từ tên pattern. [D1: RULES §§2–3; E1.coverageReview; E3.snapshots]

Mỗi báo cáo kết quả phải ghi trạng thái fixture và phạm vi SQL. Những record FAILED hoặc DEAD được tạo để thử lỗi là kết quả dự kiến của ca đó; không được gọi mọi response lỗi là PASS. Bảng ở Chương7 tách conflict hết tài nguyên, lỗi ngoài dự kiến và các bất biến sau giao dịch.

# CHƯƠNG 3 KIẾN TRÚC VÀ CÁC QUYẾT ĐỊNH

## 3.1 Kiến trúc tổng thể

Hệ thống dùng modular monolith: API và các bộ xử lý nền cùng một ứng dụng Spring Boot, nhưng phân tách trách nhiệm theo module. PostgreSQL giữ dữ liệu nghiệp vụ; Kafka phân phối các sự kiện đã commit. React gọi API qua HTTP. Trong môi trường local, backend và Vite chạy trên Windows, PostgreSQL/Kafka chạy Docker Compose; không mô tả cả bốn thành phần đều đã container hóa. [D1: ARCHITECTURE §§1–2,11]




Hình 1 Kiến trúc tổng thể từ mã nguồn

Nguồn Hình1: ARCHITECTURE.md, infra/docker-compose.yml và các module trong S1. Mũi tên event biểu diễn luồng sau commit, không phải giao dịch phân tán nguyên tử.

Quyết định giữ monolith giúp một sinh viên kiểm soát ranh giới transaction và chạy demo trên một máy. Việc dùng Kafka không bắt buộc tách microservice. Lợi ích EDA trong đồ án là dữ liệu sự kiện bền, xử lý bất đồng bộ và tái phát an toàn; chi phí là quan sát lag, thứ tự, retry, vận hành broker và quản trị record lỗi.

Ở lớp đọc, availability và dashboard là snapshot. Tại thời điểm khách bấm đặt, thông tin có thể đã đổi. API ghi luôn kiểm tra lại dưới khóa và điều kiện database; giao diện không được coi snapshot là quyền giữ vé.

## 3.2 Stack và phân chia module

| Thành phần | Phiên bản hoặc lựa chọn đã chốt |
|---|---|
| Java và framework | Temurin25.0.5+8 LTS; Spring Boot4.1.2 |
| Build và migration | Maven Wrapper3.9.16; Flyway theo BOM |
| Dữ liệu và broker | PostgreSQL17.11; Kafka4.2.2, một KRaft node |
| Frontend | React19.3.1; TypeScript7.0.2; Vite8.3.8 |
| Node và công cụ local | Node26.13.0; npm11.20.0; Docker29.9.1/Compose5.6.0 |

Các version là baseline đã được ghi nhận trong ADR0001 và evidence môi trường, không có nghĩa chúng là bản mới nhất tại mọi thời điểm. Báo cáo không đổi stack khi viết tài liệu. Kafka client/dependency cụ thể theo dependencyManagement và BOM; các bản vá thư viện nén được ghi ở ADR0007, không suy version client chỉ từ image broker.

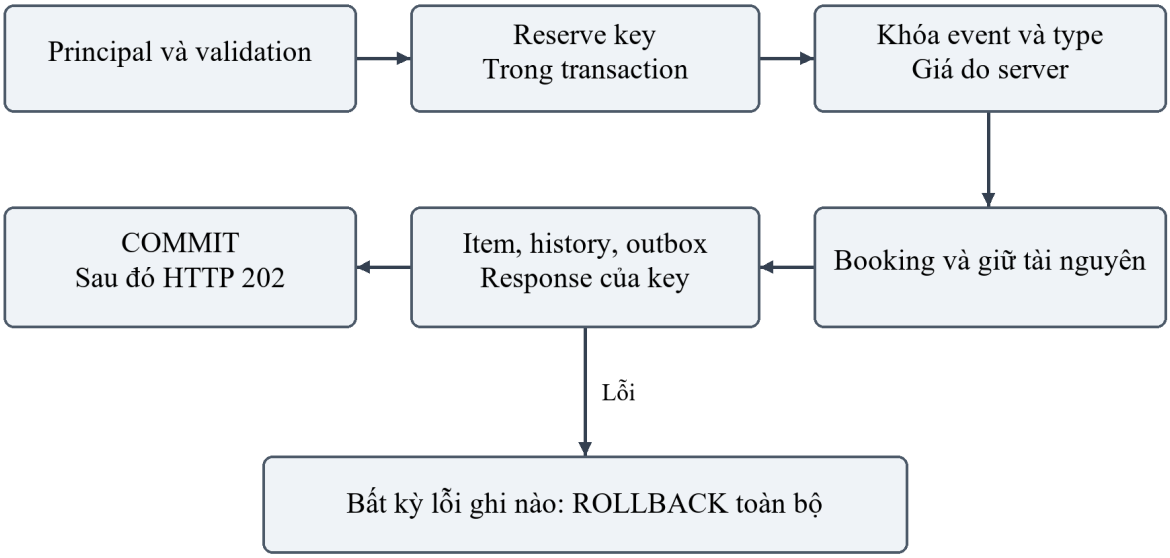
Các module identity và shared xử lý principal, HTTP policy và lỗi; events quản lý catalog; inventory thực hiện giữ/chuyển/nhả GA hoặc ghế; bookings điều phối transaction, idempotency và state. expiry quét hold đến hạn; payments ghi ledger và phân xử; tickets phát vé. messaging triển khai outbox/inbox/Kafka/failure; operations cung cấp health, metrics và recovery view.

Controller chỉ parse/validate và chuyển lệnh. BookingTransactions giữ chính sách timeout/retry; service nghiệp vụ gọi các thao tác inventory trong transaction bắt buộc. Việc chia module không ngăn mọi truy cập SQL xuyên miền bằng cơ chế máy móc; sự phụ thuộc cần được đọc cùng interface, ADR và test. Điểm quan trọng là không có bản logic phân xử payment thứ hai ở consumer hoặc expiry.

Không dùng Redis lock để quyết định tồn kho. Một database duy nhất xử lý quyền sở hữu và constraint, giúp giảm số hệ cần đồng thuận. Đổi lại, một hàng inventory GA có thể thành điểm tranh chấp khi burst cùng loại vé. Kết quả tải ở Chương7 chỉ đo cấu hình đó trên loopback; chưa đủ cơ sở để khẳng định khả năng mở rộng vô hạn.

## 3.3 Ranh giới giao dịch và thứ tự khóa

Tạo booking đặt key, kiểm tra catalog/giá, tạo booking, giữ tài nguyên, ghi item, history, outbox và response idempotency trong một transaction. Chỉ sau commit API mới trả202. Nếu một bước thất bại, request không được để lại key STARTED, item một phần hoặc outbox giả. [ADR0004–0005; S1: BookingCommandService.java]




Hình 2 Ranh giới transaction của lệnh tạo booking

Nguồn Hình2: BookingCommandService, BookingTransactions, CatalogBookingAccess, GaInventory, SeatInventory và OutboxWriter. Kafka không nằm trong khối giao dịch ghi.

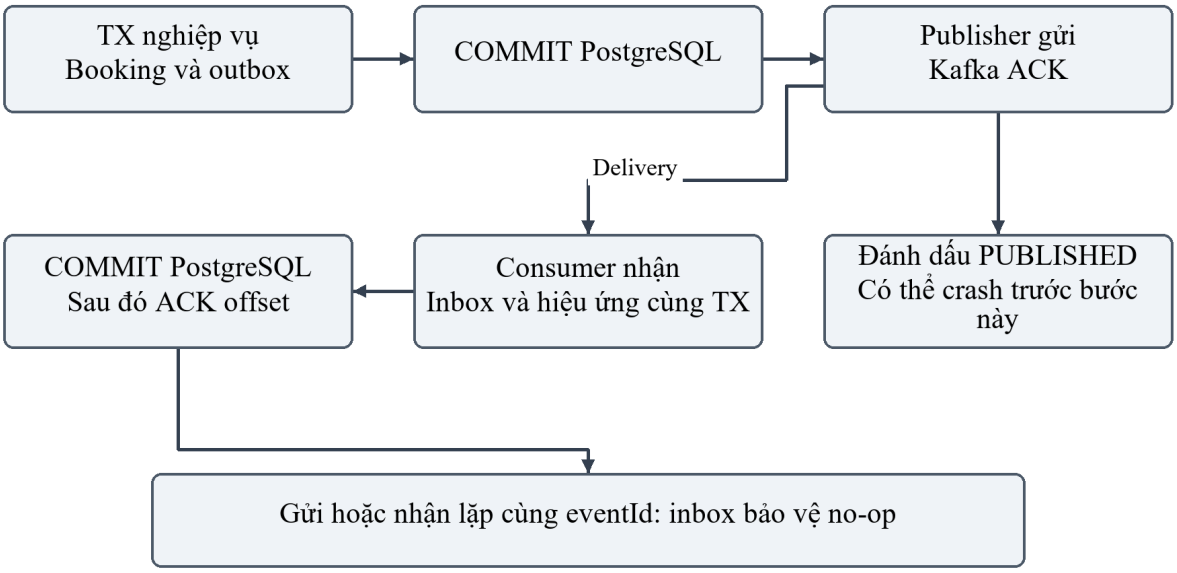
Booking mới khóa key trước, rồi event/type để giữ giá và policy ổn định. Với ghế, request chuẩn hóa UUID và khóa từng hàng theo thứ tự cố định; không suy thứ tự khóa từ một mệnh đề IN. Khi xử lý booking đã có, payment, cancel và expiry khóa booking trước, rồi inventory. Các nhánh dùng chung thứ tự để hạn chế deadlock.

Isolation ghi là READ COMMITTED với khóa và cập nhật có điều kiện. Các truy vấn nhiều bảng của booking/admin có thể dùng REPEATABLE READ để tránh tổng và danh sách thuộc hai snapshot khác nhau. Chọn isolation không thay việc kiểm tra lại điều kiện sau khóa; cũng không thay kiểm thử race. PostgreSQL mô tả các khác biệt snapshot và khóa trong tài liệu chính thức [R1–R2].

Retry toàn transaction chỉ dành cho deadlock hoặc serialization failure với số lần và deadline hữu hạn. Timeout ở key hoặc inventory có thể cùng SQLSTATE; server không đoán tất cả là key đang xử lý. Giao dịch phải kết thúc trước khi trả503 BOOKING_BUSY, để retry cùng key có thể thực hiện an toàn.

## 3.4 Dữ liệu bền và truyền sự kiện

Outbox là cầu nối giữa giao dịch PostgreSQL và Kafka. EventId, booking aggregateId, version, type, schemaVersion, occurredAt, correlationId và payload tối thiểu được lưu cùng nghiệp vụ. Publisher lấy record sau commit và gửi key theo bookingId. ACK Kafka và đánh dấu PUBLISHED trong database là hai bước; crash giữa chúng có thể làm gửi lặp. [ADR0007; S1: OutboxPublisher.java]




Hình 3 Chuỗi outbox Kafka inbox và offset

Nguồn Hình3: OutboxPublisher, EventConsumer, KafkaConfiguration và FailureStore. Các dấu COMMIT là ranh giới PostgreSQL; ACK offset là ranh giới Kafka khác.

Consumer xác thực envelope và đối chiếu liên kết dữ liệu bền. Trong cùng transaction, consumer insert inbox theo cặp consumer/eventId rồi thực hiện payment hoặc projection. Sau commit mới ACK offset. Nếu dừng sau commit trước ACK, lần nhận lại thấy inbox và không tạo hiệu ứng mới. Cùng eventId nhưng nội dung thay đổi là lỗi, không được xem như một duplicate hợp lệ.

Không có transaction hai pha gắn PostgreSQL với Kafka trong thiết kế này. Các bảo đảm delivery của broker [R3] không tự bao phủ side effect ngoài Kafka. Do đó báo cáo dùng thuật ngữ at-least-once với deduplication, và đo trạng thái nghiệp vụ trước/sau thay vì dùng tên exactly-once để suy ra kết quả.

Hai consumer có mục đích khác nhau. payment-v1 chỉ nhận PaymentResultRecorded và xử lý từ ledger; booking-audit-v1 nhận đủ event để duy trì projection thứ tự. Audit receipt không chứng minh payment đã phát vé; các receipt và booking hiện tại phải được xem riêng trên màn hình phục hồi.

## 3.5 Các lựa chọn và giới hạn kiến trúc

| Lựa chọn | Lợi ích | Đánh đổi cần quản lý |
|---|---|---|
| Modular monolith | Một codebase và transaction dễ kiểm soát | Một tiến trình có thể ảnh hưởng nhiều worker |
| PostgreSQL làm nguồn sự thật | Constraint/row lock và ledger thống nhất | Hàng GA nóng, cần đo contention |
| Outbox và inbox | Phục hồi và duplicate có dấu vết | Tăng bảng, lag, retention và vận hành |
| Retry hữu hạn | Không lặp vô hạn và không che lỗi | Cần người quản trị xử lý lỗi cuối |
| Một broker local | Demo có thể chạy trên máy cá nhân | Không có bằng chứng HA hoặc mất ổ đĩa |

Publisher dùng session advisory lock PostgreSQL để có một leader trong MVP. Đây là khóa điều phối riêng, có thể được giữ khi chờ mạng theo ADR0007; nó không phải transaction hoặc khóa booking/inventory. Mất kết nối leader hoặc ACK không rõ phải giữ event để thử lại cùng ID. Không được diễn giải lựa chọn này thành việc gọi Kafka khi đang khóa tồn kho của khách.

Consumer audit giữ last_applied_version cho từng aggregate và chỉ áp dụng version liên tiếp. Nếu có gap, record được retry rồi park. Consumer payment không áp quy tắc gap của projection, vì nó chỉ nhận một phần các event; dùng bookings.aggregate_version hiện tại để bỏ PaymentResultRecorded sẽ có thể làm mất kết quả hợp lệ.

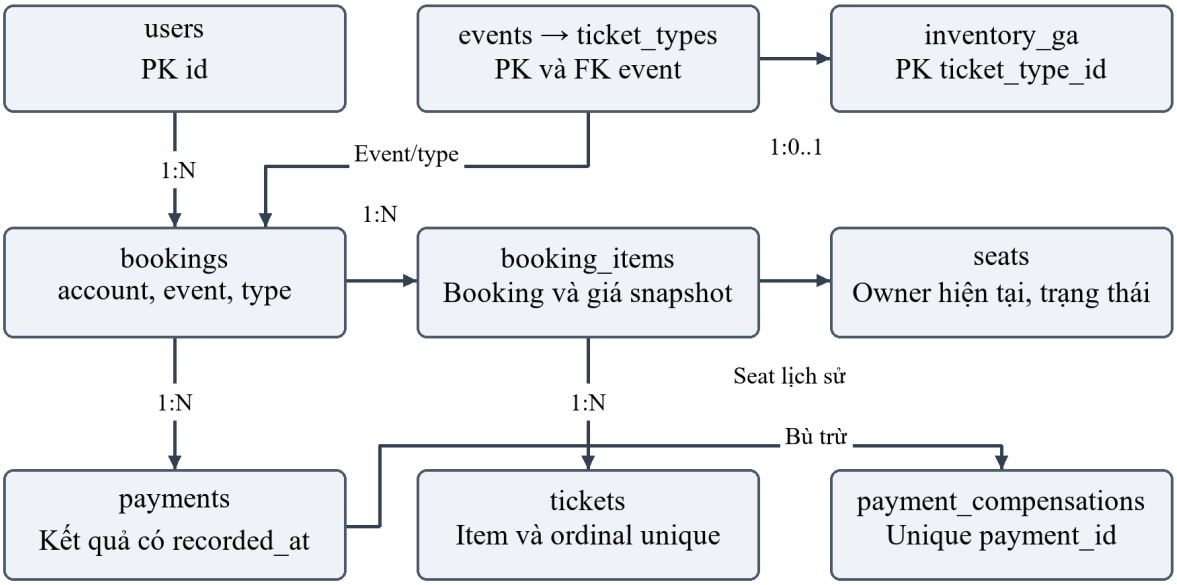
Retention broker đã cấu hình bảy ngày, nhưng MVP không tự xóa outbox, inbox hoặc key để thuận tiện audit và replay. Khi vận hành lâu hơn cần thiết kế cleanup và backup có căn cứ; không suy rằng lưu vô hạn là chính sách production phù hợp. Record poison không có outbox hợp lệ cũng không có API để sửa raw payload hoặc ép chạy.

Một số đoạn đặc tả ghi số bảng hoặc phần chưa làm ở mốc lịch sử T1/T5. Báo cáo ưu tiên migration V9, source hiện hành và evidence mới nhất khi mô tả triển khai; giữ các ghi chú lịch sử để giải thích quá trình chốt thiết kế. Điều này không biến dự kiến trong tài liệu thành tính năng đã chạy.

# CHƯƠNG 4 MÔ HÌNH DỮ LIỆU VÀ TRẠNG THÁI

## 4.1 Quan hệ dữ liệu lõi

Schema ticketing có 18 bảng nghiệp vụ sau V9; lịch sử Flyway nằm ở public. UUID được server sinh, thời gian lưu timestamptz theo UTC, tiền dùng số nguyên VND. Mô hình tách catalog, quyền phân bổ, ledger kết quả và hạ tầng event để không dùng một cột status thay cho mọi loại sự thật. [S1: db/migration/V1–V9; E4; E5.restart]




Hình 4 Quan hệ dữ liệu lõi và quyền sở hữu hiện tại

Nguồn Hình4: DATA_MODEL.md và migration V2/V5/V7/V8. Sơ đồ rút gọn; các bảng messaging/recovery được mô tả ở mục4.3. Quan hệ owner ở seats biểu diễn phân bổ hiện hành, khác lịch sử booking_items.

Một event có nhiều ticket type; mỗi type có mode GA hoặc SEATED. GA có inventory_ga, còn SEATED có các hàng seats. Một booking gắn một event/type và một account; item giữ quantity hoặc seatId cùng giá snapshot. Ticket thuộc booking và item đã xác nhận, không cấp phát một tài nguyên khác với item đã đặt.

FK tổng hợp giúp ngăn những liên kết có UUID hợp lệ nhưng sai ngữ cảnh. Bộ booking/event/type của item phải khớp booking; seat phải cùng event/type; owner hiện tại của seat phải là booking thuộc đúng event/type. Các ràng buộc này bổ sung cho kiểm tra domain khi đang giữ khóa, không thay việc kiểm tra trạng thái.

Không đặt unique(seat_id) trên toàn booking_items, vì một ghế có thể xuất hiện trong đơn đã hết hạn và đơn mới hợp lệ. Tính duy nhất hiện hành được bảo vệ ở hàng seats và được kiểm tra lại bằng truy vấn active allocation. Đây là điểm quan trọng khi giải thích vì sao lịch sử nhiều booking cùng chọn A1 không nhất thiết là bán trùng.

## 4.2 Catalog tồn kho và idempotency

| Nhóm bảng | Khóa hoặc điều kiện quan trọng |
|---|---|
| users | PK id, demo_role CUSTOMER/ADMIN; không lưu token |
| events và ticket_types | Sale window hợp lệ; type thuộc event; giá không âm |
| inventory_ga | PK ticket_type_id; held/sold không âm; tổng không vượt capacity |
| seats | Unique(event_id,label); state/owner/deadline nhất quán |
| bookings | Account/event/type; payment_intent_id unique; sáu state |
| booking_items | FK tổng hợp; quantity 1..6; tổng giá bằng quantity×giá đơn vị |
| idempotency_keys | PK(account,operation,key); booking_id unique; SHA-256 request |

State AVAILABLE của seat đi cùng owner và deadline NULL. HELD bắt buộc owner/deadline; SOLD vẫn có owner nhưng deadline NULL. Có thể đếm các trạng thái để trình bày sơ đồ, nhưng không đưa owner của người mua ra public availability. Version của seat hỗ trợ quan sát các lần cập nhật, không phải thứ thay thế row lock trong thuật toán hiện có.

Idempotency reserve key trước khi tạo booking nên booking_id và response có thể NULL tạm trong transaction. Deferred constraint trigger của V5 kiểm tra trạng thái cuối ở COMMIT: key chỉ được lưu khi booking và response 202 hoàn chỉnh, bookingId trong JSON khớp booking. Vì vậy không có bản ghi STARTED bền cần cleanup riêng hoặc lease để phục hồi.

V6 bổ sung kiểm tra history ban đầu vì CHECK PostgreSQL chấp nhận biểu thức UNKNOWN; không sửa ngược migration đã áp dụng. V4 cũng siết điều kiện JSON bookingId để không chấp nhận JSON null. Hai trường hợp cho thấy viết một biểu thức CHECK trông hợp lý chưa đủ; cần negative test trên PostgreSQL thật. [ADR0003–0004]

Chỉ mục phục vụ expiry theo status/expires_at, lịch sử tài khoản theo created_at/id, ghế theo event/type/status và truy vấn idempotency theo PK. Mô hình không thêm index cho mọi cột theo thói quen; lựa chọn phải gắn với đường truy vấn và chi phí ghi. Kết quả hiện tại chưa có phân tích execution plan dài hạn hoặc kích thước dữ liệu thương mại.

## 4.3 Ledger vé outbox và dữ liệu phục hồi

| Bảng | Vai trò và ràng buộc cốt lõi |
|---|---|
| payments | Unique(booking_id,request_id); recorded_at của DB; processed_at |
| tickets | Unique(item,ordinal), unique(code); FK đúng booking/item |
| payment_compensations | Unique(payment_id), một nghĩa vụ cho một kết quả |
| booking_status_history | Cạnh state, thời điểm và cause_event_id |
| outbox_events | Unique(aggregate_id,aggregate_version), envelope và trạng thái gửi |
| inbox_events | PK(consumer,event_id), envelope_hash và thời điểm xử lý |
| event_projections và projection_events | Version đã áp dụng và lịch sử audit riêng |
| message_failures | Vị trí Kafka, raw record, attempts, DEAD/RESOLVED, trạng thái DLQ |
| compensation_attempts | PK(compensation_id,attempt), kết quả refund mô phỏng |

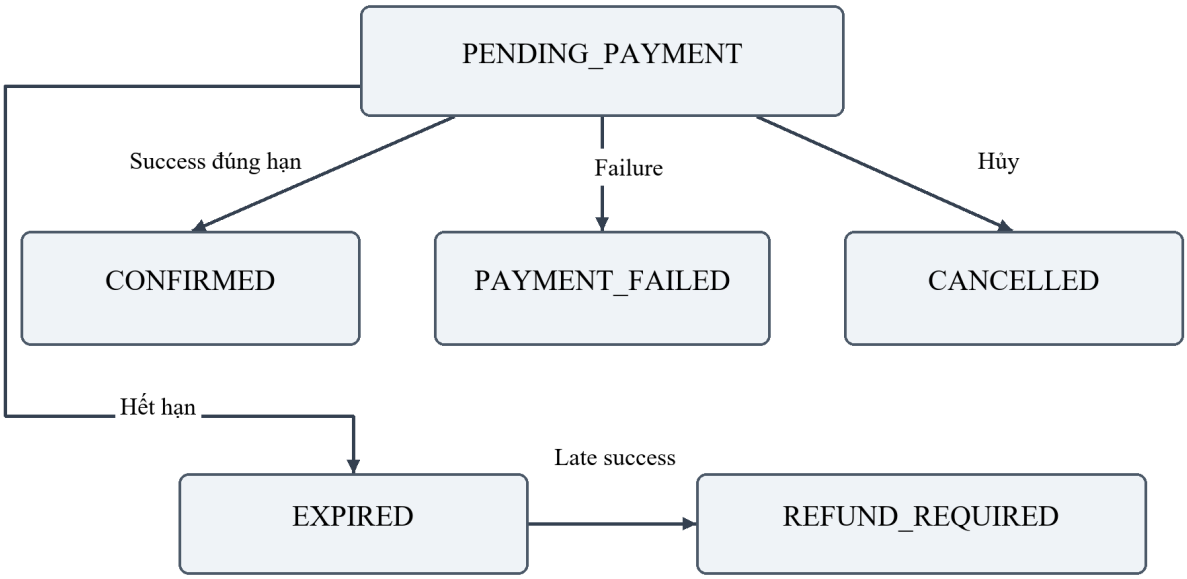
payments.id là paymentResultId của một lần ghi nhận; payment_intent_id thuộc booking là định danh nhóm thanh toán; request_id là khóa chống gửi lặp của client. Ba ID không được dùng lẫn. Kết quả SCHEDULED có due_at nhưng recorded_at NULL. SUCCESS hoặc FAILURE bền phải có recorded_at; processed_at cho biết bộ phân xử đã xử lý ledger đó.

Outbox có schema_version, payload JSON, correlation_id và thời điểm sự kiện. attempts, next_attempt_at, last_error và published_at phục vụ vận hành. replay_requests giữ tối đa ba audit gồm ADMIN, requestId, reason và requestedAt. Replay thay trạng thái gửi có kiểm soát nhưng không đổi eventId hoặc payload nghiệp vụ.

Inbox liên kết logic với event, không FK bắt buộc tới outbox vì vòng đời lưu trữ độc lập. Mỗi consumer có receipt riêng; unique toàn cục chỉ trên eventId sẽ chặn consumer thứ hai. Projection audit cũng không là nguồn để phát vé hoặc điều khiển tồn kho.

Raw record lỗi được giữ trong failure DB để điều tra, nhưng API recovery chỉ trả metadata và lý do an toàn. Lưu raw ở database không đồng nghĩa được phép in nó vào console, báo cáo hoặc ảnh. Truy vấn báo cáo chọn counter, ID và hash thay vì xuất payload payment hoặc mã vé thật của fixture. [D1: DATA_MODEL V9; ADR0007; E3]

## 4.4 State machine của booking và tài nguyên




Hình 5 State machine booking theo PRD

Nguồn Hình5: PRD §5 và docs/contracts/booking-transitions.json. Chỉ EXPIRED có cạnh sang REFUND_REQUIRED; không có cạnh từ REFUND_REQUIRED trở lại CONFIRMED.

PENDING_PAYMENT biểu thị quyền giữ tạm thời. Khi success đúng hạn được phân xử và quyền giữ còn hợp lệ, tài nguyên chuyển sang đã bán, vé được phát và booking CONFIRMED. Failure, cancel hoặc expiry thắng chỉ nhả tài nguyên còn thuộc booking đó và ghi trạng thái terminal tương ứng. State và hiệu ứng inventory nằm cùng transaction.

Nếu SUCCESS tới sau khi EXPIRED đã commit, booking chuyển REFUND_REQUIRED và tạo nghĩa vụ bù trừ. Nếu booking còn pending nhưng recorded_at đã sau deadline, processor thực hiện EXPIRED/release rồi REFUND_REQUIRED trong cùng giao dịch, giữ đủ hai mốc history. Không được bỏ mốc hết hạn để làm lịch sử đơn giản hơn.

Success sau CANCELLED hoặc PAYMENT_FAILED có thể tạo compensation nhưng giữ nguyên state; không tự áp cạnh REFUND_REQUIRED cho các trạng thái không có cạnh đó. Failure tới sau CONFIRMED không được giải phóng vé đã bán. Booking REFUND_REQUIRED vẫn giữ state sau compensation COMPLETED, vì trạng thái booking và tiến độ bù trừ là hai khái niệm khác nhau.

Ghế có vòng đời AVAILABLE→HELD→SOLD hoặc HELD→AVAILABLE. GA dùng bộ đếm held/sold. Thời gian qua hạn không tự sửa bất kỳ state nào; cần worker hoặc processor commit chuyển trạng thái. Nhờ đó request mới không thể "cướp" ghế đã quá giờ nhưng worker chưa hoàn tất việc nhả. [ADR0005–0007; S1: BookingLifecycle, InventoryRelease, PaymentResultProcessor]

## 4.5 Migration và dữ liệu mẫu tái lập

| Migration | Mục đích trong tiến trình triển khai |
|---|---|
| V1 | Tạo schema ticketing |
| V2 | Catalog, booking/item nền, seat và inventory GA |
| V3 | Payment ledger và outbox |
| V4 | Siết JSON bookingId trong outbox |
| V5 | Idempotency và lịch sử booking |
| V6 | Siết transition history ban đầu |
| V7 | FK tổng hợp owner ghế cùng event/type |
| V8 | Vé, compensation và correlation payment |
| V9 | Inbox, projection, failure/DLQ và lịch sử refund |

Các file đã áp dụng được giữ nguyên; thay đổi schema đi qua version mới. Flyway so migration với schema history khi startup. Không dùng sửa checksum hoặc xóa lịch sử như một cách làm cho ứng dụng chạy. Seed nằm ngoài migration và chỉ bật local/test, theo INSERT ON CONFLICT DO NOTHING; khởi động lại không ghi đè inventory hoặc giá đã được thao tác. [S1; R7]

Seed cung cấp hai mode sự kiện và customer/admin demo. Fixture kiểm thử không lấy các con số seed bất biến làm assertion tổng database, vì người dùng và test khác có thể tạo dữ liệu. Ca phân trang phải nhận diện các booking chính nó tạo, kiểm thứ tự và ID không trùng ngay cả khi có dữ liệu khác.

T9.4 và T10.1 đã kiểm database riêng từ sạch, V1–V9 thành công, seed và khởi động lại. T10.2 tạo project riêng, thêm fixture mới từng lượt và dùng SQL chỉ đọc để so hash 18 bảng cùng chín dòng Flyway trước/sau Stop/Start. Đây là dừng có kiểm soát với dữ liệu đã ổn định; không phải thử mất điện đúng lúc COMMIT. [E4; E5.restart; E6.restart]

Giữ volume khi dừng là một phần của tính tái lập: người khác có thể chạy lại mà không mất booking đã demo. Tạo môi trường sạch để thử không đồng nghĩa được xóa volume demo cũ. Script phải xác định project, cổng và đường dẫn sở hữu trước khi dừng tiến trình hoặc container.

# CHƯƠNG 5 TRIỂN KHAI CÁC LUỒNG NGHIỆP VỤ

## 5.1 Giữ và bán vé GA

GaInventory.reserve dùng một UPDATE có điều kiện. PostgreSQL kiểm tra và ghi tăng held_count trong cùng câu lệnh, tránh tách đọc tồn và ghi tồn thành hai thao tác không liên kết. Nếu không có đúng một hàng thay đổi, service trả conflict và transaction booking rollback. [S1: inventory/GaInventory.java]

```
UPDATE ticketing.inventory_ga
SET held_count = held_count + :quantity
WHERE ticket_type_id = :type_id
  AND held_count + sold_count + :quantity <= capacity;
```

Đoạn trên là trích lược có đặt tên tham số để giải thích, không phải một API mới. Domain đã kiểm quantity hợp lệ, catalog đang bán và type đúng mode. Trong transaction còn có booking, item snapshot, key, history và BookingHeld; UPDATE thành công riêng lẻ không đủ để xác nhận command đã hoàn tất.

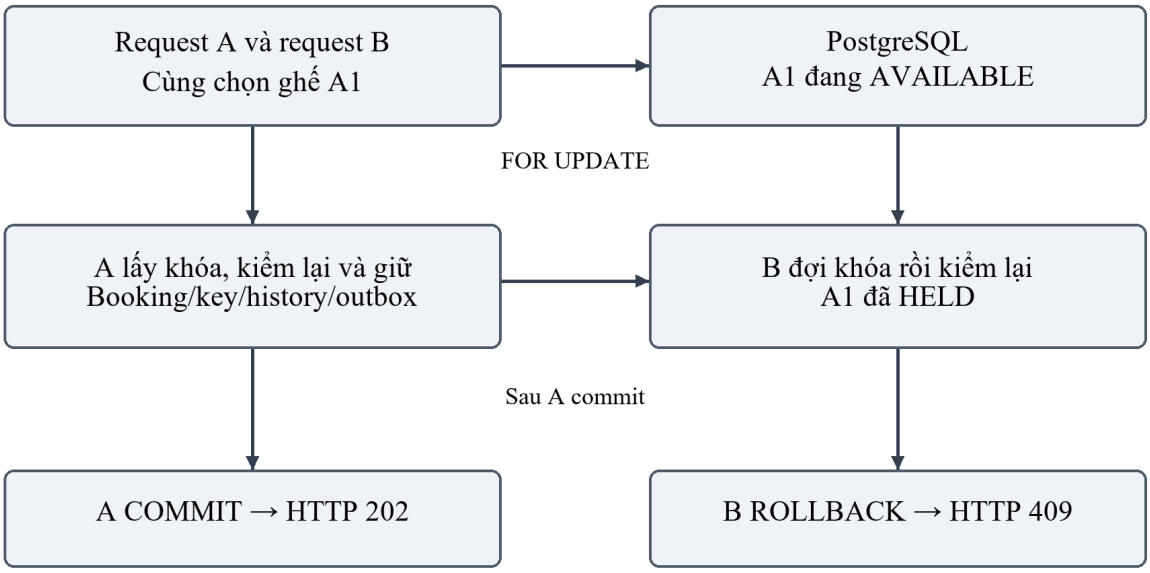
Khi payment thành công hợp lệ, processor khóa booking, xác minh quyền giữ và chuyển đúng quantity từ held sang sold. Khi failure/cancel/expiry, chỉ quantity còn thuộc booking PENDING được nhả. Các lần xử lý sau khi terminal là no-op về inventory. Không đặt lại sold_count từ một snapshot API hoặc từ tổng tính ở client.

Một hàng GA giúp kiểm soát invariant rõ ràng nhưng tạo hàng nóng khi mọi khách chọn cùng type. Pool database có thể nhỏ hơn số HTTP worker; request sẽ chờ connection hoặc row lock trong ngân sách đã chốt. Bài đo T9.2 cố tình dùng capacity nhỏ và 50 request để quan sát conflict dự kiến, không phải tối đa hóa số booking thành công.

Giá tổng là số nguyên được server tính với kiểm tra giới hạn. booking_items giữ giá tại lúc đặt, nên thay đổi catalog sau đó không sửa đơn cũ. Availability trả số tham khảo còn lại theo capacity-held-sold; chính conditional UPDATE là điểm quyết định giữ, không phải con số trên giao diện.

## 5.2 Ghế định danh và đặt nhiều ghế nguyên tử

SeatBookingRequest chuẩn hóa UUID, kiểm tra trùng và thứ tự. SeatInventory khóa từng ID, đọc lại event/type/status rồi chỉ chuyển AVAILABLE sang HELD. Booking được insert trước để FK owner có đích, nhưng toàn bộ vẫn chưa commit. Nếu ghế thứ hai không hợp lệ hoặc đang bị giữ, mọi thay đổi ghế trước đó, booking và key đều rollback.




Hình 6 Hai request cạnh tranh cùng ghế

Nguồn Hình6: SeatInventory, BookingCommandService và PostgresSeatExpiryTest. Request B chỉ thắng ở lượt sau nếu tài nguyên đã được nhả hợp lệ; không tự chiếm hold quá giờ.

Khóa theo ID bất biến tạo một thứ tự chung giữa các request chọn các tập ghế giao nhau. Chọn A1,A2 và A2,A1 phải được chuẩn hóa về cùng thứ tự, tránh phụ thuộc thứ tự người bấm. Một SELECT/UPDATE nhiều ID không tự là bằng chứng execution plan khóa theo thứ tự mong muốn; implementation hiện khóa rõ từng hàng.

Hold quá hạn vẫn là HELD trước khi worker commit. API có thể trả HOLD_EXPIRY_PROCESSING để client tải lại; không có nhánh cleanup inline trong tạo booking. Sau khi ghế được cấp cho booking mới, retry của worker hoặc event cũ phải kiểm owner, không được nhả quyền mới dựa vào seatId trần.

T9.2 thử 50 request cùng chọn cặp ghế, chỉ một booking giữ cả hai. T10.2 minh họa nhẹ hơn: hai request cùng persona tranh A1, một202/một409 rồi payment request thắng. Hai tình huống có số request và allocation khác nhau; báo cáo không trộn số vé hoặc gọi chúng là hai phép đo tương đương. [E2.measurements; E6.main.seatContest]

## 5.3 Idempotency cho booking và payment

Scope key booking là account/operation/key. Hash canonical được tính từ UUID chuẩn hóa và quantity, hoặc danh sách seatId đã sắp. Cùng key, cùng nội dung trả lại body202 và bookingId đã lưu; cùng key nhưng nội dung khác trả409 IDEMPOTENCY_KEY_REUSED. Việc đảo thứ tự danh sách ghế không tạo thao tác khác. [ADR0004–0005]

```
BEGIN
  INSERT key ON CONFLICT DO NOTHING
  nếu key đã có: SELECT hash và response đã commit
  nếu hash khác: lỗi409
  nếu key mới: thực hiện booking và hoàn thiện response
COMMIT trước khi trả HTTP202
```

Unique insert giúp request trùng đang chạy chờ giao dịch trước. Ở READ COMMITTED, statement tiếp theo đọc kết quả đã commit. Nếu giao dịch đầu rollback, request sau có thể thực hiện lại; không phải xóa key lỗi bằng một job dọn STARTED. Timeout có thể phát sinh ở unique key hoặc inventory và được rollback trước khi trả503.

Response lưu là kết quả của lệnh tạo ban đầu, không phải projection liên tục của booking. Sau khi booking CONFIRMED hoặc EXPIRED, gửi lại cùng key vẫn có thể trả snapshot PENDING_PAYMENT ban đầu; client cần GET booking để biết trạng thái hiện tại. T10.2 đã kiểm chính hành vi này, không sửa response cũ để làm minh họa dễ hiểu hơn. [E6.main.gaIdempotency]

Payment simulator có scope booking/requestId/scenario. Cùng requestId và scenario trả kết quả chấp nhận cũ; đổi scenario bị conflict. Replay sự kiện có requestId/reason riêng và audit riêng, không dùng chung bảng key booking. Một key không có ý nghĩa “chống trùng toàn hệ thống” nếu thiếu scope nghiệp vụ.

Frontend lưu lệnh trước khi gửi trong sessionStorage và giữ nguyên khi mạng lỗi hoặc503. Không tự tạo key mới sau response mơ hồ, không tự POST khi reload/chuyển persona. Đây là hỗ trợ cho protocol server; việc khóa nút double click không thay idempotency database hoặc test request đồng thời.

## 5.4 Expiry Worker và nhả hold

Expiry Worker là tác vụ nền chủ động, không phụ thuộc việc có đơn mới. Scheduler mặc định quét mỗi năm giây, tối đa 50 candidate; một candidate được xử lý trong transaction riêng. Chọn candidate không giữ một batch transaction xuyên nhiều booking. [S1: expiry/ExpiryService.java; ADR0005]

Sau khi khóa booking bằng FOR UPDATE SKIP LOCKED, worker kiểm lại PENDING_PAYMENT và deadline theo clock_timestamp của PostgreSQL. Worker gọi PaymentResultProcessor trên ledger bền trước khi nhả. Nếu state đã đổi vì payment hợp lệ, worker không expire. Nếu vẫn pending và đã đến hạn, InventoryRelease cùng transition EXPIRED/history/outbox commit nguyên tử.

| Tình huống | Quyết định |
|---|---|
| Chưa đến hạn hoặc đã terminal | Không thay đổi inventory |
| Booking đang bị khóa | Bỏ qua/lượt sau, không nhận quyền của thread khác |
| Có timely SUCCESS bền | Processor phân xử trước, có thể CONFIRMED |
| Không có kết quả cứu hold hợp lệ | Nhả đúng tài nguyên đang sở hữu và EXPIRED |
| Ghi history/outbox lỗi | Rollback cả nhả và transition; lượt sau thử lại |

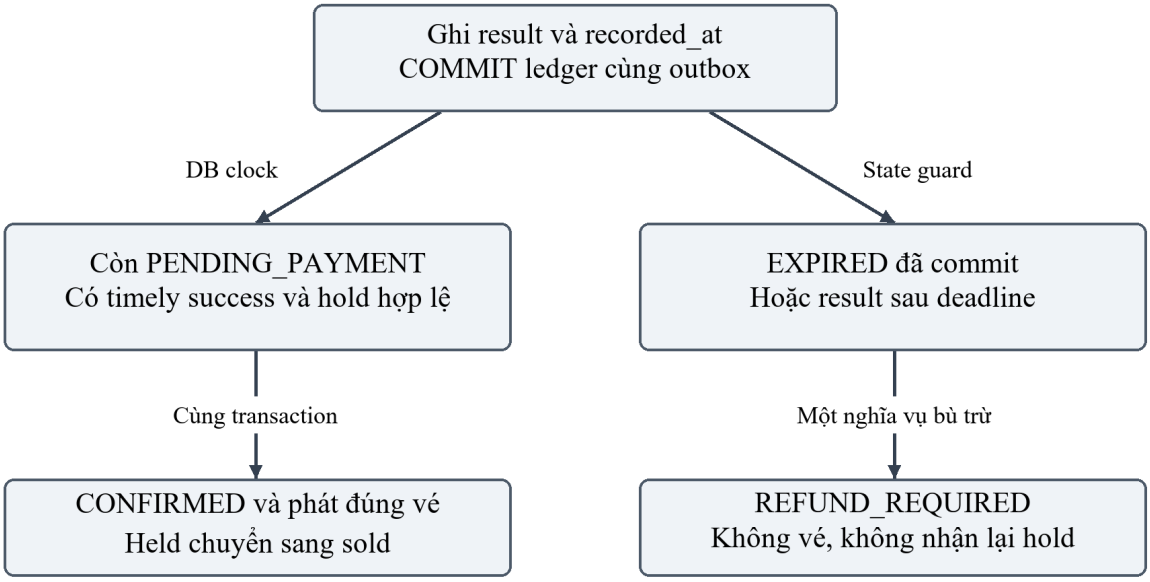
Nhiều worker có thể thấy cùng candidate, nhưng row lock và recheck làm chỉ một transition hiệu lực. Retry hoặc restart quét lại dữ liệu bền, không cần lease trong RAM hoặc Redis. Booking mới chờ worker commit nhả; đến giờ TTL trên trình duyệt không làm ghế AVAILABLE.

Ca metric đã dừng TaskScheduler thật, đợi hold quá hạn rồi đo overdue tăng; khi resume, worker tự nhả và metric giảm theo SQL. Ca concurrency cũng chạy nhiều worker/lặp để kiểm một EXPIRED history và một BookingExpired. Những kiểm tra này khác với việc gọi expire trực tiếp trong một unit test; báo cáo nêu rõ loại kiểm chứng khi mô tả sự cố. [E1.coverageReview; TEST_PLAN Gate T8.2]

Worker chậm, batch bận hoặc database lỗi có thể khiến hold quá giờ chưa đặt lại được ngay. Thiết kế ưu tiên không cấp phát hai lần; metric overdue/oldest age và khả năng thử lại giúp người vận hành nhận biết độ trễ, không che nó bằng một trạng thái giả trên UI.

## 5.5 Thanh toán mô phỏng và phân xử với expiry

API simulate-payment nhận SUCCESS, FAILURE hoặc DELAY_TIMEOUT cùng requestId; không nhận số tiền, user, status tùy ý hoặc timestamp client. Kết quả được ghi cùng PaymentResultRecorded dưới booking lock bằng đồng hồ DB. Ghi result và xử lý result là hai transaction có chủ đích; lỗi xử lý không xóa một kết quả đã commit. [ADR0006]




Hình 7 Hai nhánh phân xử payment và expiry

Nguồn Hình7: PaymentResultProcessor và ExpiryService. Tiêu chí đúng hạn là recorded_at≤expires_at; sau EXPIRED đã commit không có đường nhận lại tài nguyên.

Khi Kafka bật, simulator trả202 sau ledger/outbox; consumer sẽ phân xử bất đồng bộ. Nếu timely result đã bền nhưng Kafka đang ngừng, expiry worker đọc result đó và có thể confirm thay vì nhả. T9.3 kiểm broker thật ngừng, inbox payment vẫn0 nhưng worker phát đúng hai vé; sau phục hồi inbox1 và không phát thêm. [E3.comparisonProofs]

DELAY_TIMEOUT lưu SCHEDULED với due_at sau expires_at một giây, chưa có recorded_at; maintenance chỉ ghi SUCCESS khi tới hạn, không sleep giữ lock. Nếu scheduler trễ, recorded_at vẫn là lúc ghi bền thực tế, không backdate theo due_at. Sau khi mất hold, success tạo REFUND_REQUIRED/compensation chứ không tự chiếm lại ghế của khách khác.

Trong transaction phân xử, một timely success đang bền được ưu tiên trước failure chưa xử lý; trạng thái terminal đã commit không đảo ngược để confirm. Khi phối hợp record/process ở adapter đồng bộ, hai bước chia sẻ deadline tổng, không mỗi bước thêm mười giây mới. Các ca rollback và cạnh deadline được thử riêng; một kết quả202 không tự là bằng chứng khách đã có vé.

## 5.6 Phát vé và bù trừ mô phỏng

Vé được phát trong transaction CONFIRMED, cùng chuyển held sang sold, history, outbox và processed_at. Một item GA quantity N tạo N vé với ordinal1..N; mỗi item ghế tạo một vé ordinal1. Unique(item,ordinal) và unique(code) chống phát trùng, còn domain kiểm trạng thái và quantity. BookingConfirmed là thông báo, không phải lệnh phát vé lần hai. [S1: tickets/TicketService.java; ADR0006]

Mã vé là định danh opaque do server sinh và lưu; API owner/ADMIN trả vé cùng ordinal hoặc seatId/seatLabel. Đây là dữ liệu tra cứu demo, không phải QR có chữ ký, chứng chỉ thanh toán, cơ chế quét offline hoặc endpoint check-in production. Báo cáo không bổ sung khả năng chưa có trong task.

| Kết quả phân xử | Vé và compensation |
|---|---|
| Timely SUCCESS, hold hợp lệ | Đúng số vé đã chọn; không tạo nghĩa vụ hoàn |
| FAILURE đang pending | Không vé; nhả hold, PAYMENT_FAILED |
| SUCCESS sau EXPIRED | Không vé; REFUND_REQUIRED và một nghĩa vụ/payment |
| SUCCESS sau CANCELLED/PAYMENT_FAILED | Không confirm; bù trừ, giữ state hiện tại |
| Duplicate xử lý hoặc BookingConfirmed | Không phát lại hoặc hoàn tất bù trừ lần hai |

RefundRecoveryService chỉ mô phỏng bằng cập nhật DB, không gọi cổng thanh toán hay chuyển tiền. Worker khóa cùng nghĩa vụ, kiểm ledger, ghi attempt và trạng thái trong một transaction; tối đa ba attempt, có backoff. COMPLETED là kết quả cuối không chạy lại. Sau hết ngân sách, FAILED được giữ để điều tra, không tạo row nghĩa vụ khác để “thử lại từ đầu”.

Trong T9.3, ca late success có một nghĩa vụ, hai attempt sau một lỗi mô phỏng, chỉ một completion; chạy 12 attempt đồng thời không tạo hai lần hoàn tất. T10.2 dùng cấu hình không gây refund failure nên có một attemptCOMPLETED. Hai số attempt thuộc fixture khác nhau, không mâu thuẫn. [E3.comparisonProofs; E6.main.late]

## 5.7 Outbox publisher sau commit

OutboxWriter yêu cầu một transaction đang có và kiểm version booking. Event được lưu bền trước khi publisher có thể nhìn thấy. Publisher gửi theo bookingId, chỉ chọn version thấp nhất chưa PUBLISHED cho mỗi booking; version sau không được vượt version đangFAILED. [S1: messaging/OutboxWriter.java, OutboxPublisher.java]

```
Đọc event đủ điều kiện và nhận quyền publisher
Gửi envelope gốc với key bookingId rồi chờ ACK
ACK rõ ràng → đánh dấu PUBLISHED trong DB
Lỗi hoặc ACK không rõ → giữ ID, retry có giới hạn
Hết ba attempt → FAILED, giữ record để điều tra/replay
```

Publisher không giữ khóa booking hoặc inventory khi gọi Kafka. Session advisory lock điều phối leader là cơ chế riêng. Mỗi chu kỳ gửi tối đa ba lần với backoff một rồi hai giây. Kafka client có timeout/ACK semantics riêng; số lần network retry bên trong producer không tự đồng nghĩa attempts nghiệp vụ của outbox.

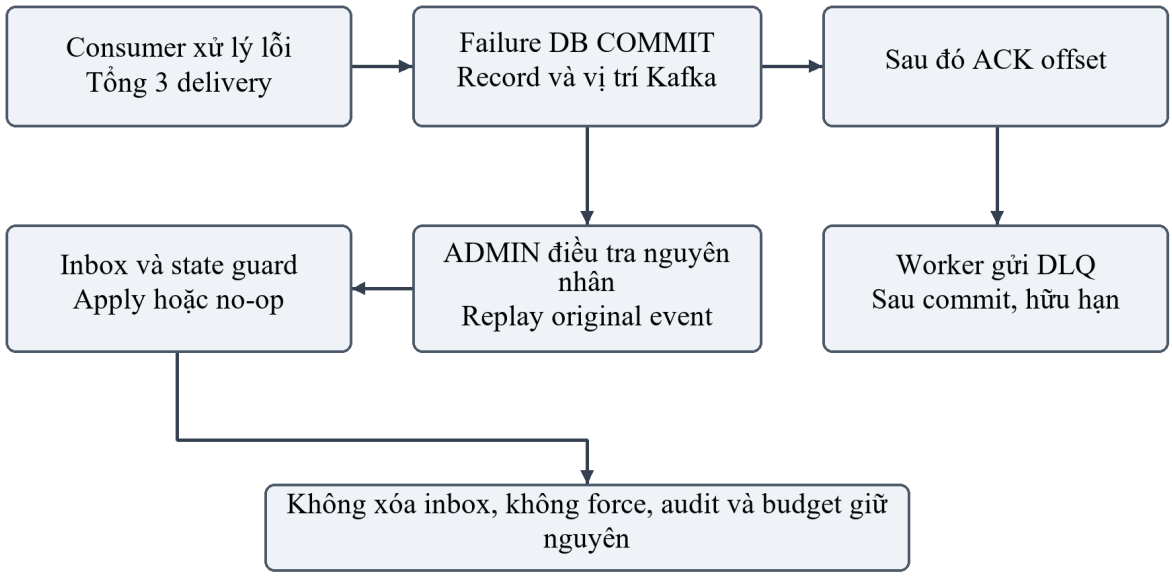
Cửa sổ sau Kafka ACK nhưng trước DB mark được kiểm bằng fault injection: việc đánh dấu rollback khiến event còn chờ và gửi lại cùng eventId. Consumer đã commit inbox sẽ no-op. Không có đoạn code nào xóa event khi chưa rõ đã gửi; chọn "có thể gửi lại" tốt hơn làm mất kết quả đã commit.

Khi Kafka outage kéo dài đến FAILED, bật broker chưa tự làm mọi event terminal gửi lại. ADMIN cần xác định nguyên nhân và replay version đầu của booking đó theo audit. Sau khi version đầu publish, version sau mới tiến triển. Runbook T10.2 minh họa create/cancel khi broker tắt để inventory về trạng thái ổn định trong khi outbox còn chờ. [E3.comparisonProofs; E6.main.outage/recovery]

Các record outbox cũ và mới cùng tồn tại; truy vấn kiểm thử phải lọc aggregate của fixture, không lấy tổng pending toàn database để kết luận một request thành công hay thất bại. Tuổi outbox gồm PENDING lẫn FAILED vì cả hai đều là công việc chưa hoàn tất.

## 5.8 Consumer retry DLQ và dữ liệu lỗi bền

Consumer kiểm event type, schema, UUID, key, aggregate và payload liên kết với row bền. payment-v1 tra paymentResultId thuộc booking và terminal result hợp lệ; không lấy status/timestamp từ một message tùy ý để phát vé. Audit consumer giữ thứ tự projection, không tự thay booking.




Hình 8 Lỗi consumer và đường phục hồi có kiểm soát

Nguồn Hình8: EventConsumer, FailureStore, KafkaConfiguration, ReplayService và ADR0007. DLQ là thông báo sau khi record lỗi đã commit vào PostgreSQL.

Delivery lỗi được thử tổng cộng ba lần tại partition, không dùng retry topic riêng gây đảo thứ tự. Sau lỗi cuối, FailureStore lưu nguyên record, key và vị trí consumer/topic/partition/offset trước khi ACK. Nếu DB không ghi được failure, consumer không ACK. Worker gửi DLQ sau commit cũng có retry hữu hạn. [S1; R4]

Inbox insert và hiệu ứng nghiệp vụ cùng rollback nếu handler lỗi. Một receipt được ghi rồi giao dịch thất bại không được trở thành lý do bỏ payment ở lượt sau. Ca T9.3 giữ0inbox/0 vé sau ba delivery lỗi, rồi replay sau khi sửa nguyên nhân tạo đúng1inbox/2 vé và failureRESOLVED. Poison record tạo failure/DLQ riêng cho hai consumer; không lấy tổng cố định khi fixture khác còn tồn tại. [E3.comparisonProofs]

DLQ không phải API để tự động phát lại mọi message. Message malformed không có outbox hợp lệ phải được điều tra; giao diện không cho sửa raw record hoặc bỏ state guard. Việc đọc failure an toàn và replay original event có audit là một phần chức năng vận hành, không phải đường vòng để vượt các bất biến nghiệp vụ.

## 5.9 Replay có quyền và audit

Replay API chỉ ADMIN local/test, chọn từng event và có lý do. Service khóa outbox, kiểm requestId/reason, append audit rồi đặt lại PENDING trong một transaction. Tối đa ba request khác nhau/event; cùng requestId/reason trả kết quả cũ, không reset lần nữa hoặc trừ budget mới. Đổi reason cùng requestId bị409. [S1: operations/ReplayService.java]

| Điều kiện | Kết quả bắt buộc |
|---|---|
| Anonymous hoặc token không hợp lệ | 401 |
| CUSTOMER gọi thao tác ADMIN | 403 |
| Không có event | 404 |
| Event đã xử lý, replay hợp lệ | Có thể202 và consumerno-op |
| Request thứ tư khác nhau | 409 REPLAY_LIMIT_REACHED |
| Gửi lại request cũ khi budget0 | Trả kết quả cũ theo policy, không bypass |

Replay giữ nguyên envelope/eventId, không xóa inbox. Một event đã publish và xử lý thành công có thể được phát lại để kiểm tính lặp an toàn, nhưng không được tạo payment/vé/history mới. Nếu projection gặp gap, phải replay version thiếu trước rồi event cao hơn; giao diện không có nút ép bỏ qua version.

Recovery view cho xem trạng thái publisher, receipt từng consumer, failure metadata, lý do an toàn và lịch sử audit. Panel mở gần vùng thao tác và focus vào trường lý do; đóng rõ ràng. Chọn hoặc tải lại không tự POST. Nếu response mất sau202, UI giữ requestId/reason để người dùng gửi lại đúng request, kể cả ngân sách request mới đã hết.

T10.2 replay PaymentResultRecorded đã xử lý, gửi haiHTTP cùng request. Audit chỉ có một row, budget còn hai, log có DUPLICATE ở payment-v1 và booking-audit-v1; hash nghiệp vụ/inbox/projection trước và sau giống nhau. Những thay đổi hợp lệ ở trạng thái gửi hoặc audit outbox không nằm trong hash nghiệp vụ dùng cho phép so này. [E6.main.replay]

## 5.10 Giao diện logging metrics và health

Giao diện customer có catalog, chọn quantity hoặc ghế, countdown, mô phỏng payment, lịch sử và vé. Màu ghế đi cùng nhãn; không dựa màu đơn lẻ. Giá, tổng tiền và trạng thái lấy từ API; countdown về0 vẫn phải chờ server. Customer hiện đọc tối đa100sự kiện/100 booking, không phải UI phân trang vô hạn. Admin có phân trang, filter trạng thái, catalog editor, inventory và recovery.

Không dùng wireframe thay ảnh đã chạy. Báo cáo mô tả màn hình từ DESIGN và source; xác nhận Chrome ở E5 là lời người dùng về GA1 vé/A1đúngghế và hai booking trong history/dashboard. DOM/live HTTP đã chạy không tự chứng minh mọi viewport/mobile hoặc khả năng tiếp cận trên browser thật. [E1; E5.chromeConfirmation; E6.evidence.chrome]

| Nhóm quan sát | Nội dung và đơn vị |
|---|---|
| Log cấu trúc | correlationId, bookingId, messageeventId, stage/outcome |
| Booking và overdue | Gauge sốbooking; tuổi quáhạn lớn nhất tínhgiây |
| Outbox | Gauge sốmessage theo state; tuổi chưa publish tínhgiây |
| Retry và DLQ | Counter retry/delivery; gaugefailure và trạngtháiDLQ |
| Độ trễ API | http.server.requests timer, đơn vigiây |
| Snapshot metric | available0/1 và tuổi snapshot; lỗi không giả thành0 booking |

Label chỉ dùng tập giá trị hữu hạn như state, consumer hoặc route template; không đưa bookingId, token, account hay rawURI vào label. Counter/timer theo tiến trình reset khi JVM restart; gauge/recovery DB phản ánh trạng thái bền. Timer chưa bật histogram/p95; phân vị tải trong Chương7 do công cụ đo HTTP tính từ mẫu riêng, không được gán cho Actuator. Theo dấu hiện dùng correlation ID và log, chưa có bằng chứng triển khai backend trace/span OpenTelemetry.

Root/readiness phản ánh PostgreSQL và Kafka khi messaging bật; broker tắt làm503/DOWN, liveness vẫn200 nếu tiến trình sống. Health không kiểm tra toàn bộ backlog hoặc mọi partition có ghi được. Kết hợp health với tuổi outbox, overdue và receipt mới giúp phân biệt ứng dụng sống với nghiệp vụ đang tiến triển. [S1: operations; D1: ARCHITECTURE T8.1–T8.2; R5]

# CHƯƠNG 6 PHƯƠNG PHÁP KIỂM THỬ

## 6.1 Các lớp kiểm tra và môi trường

Phương pháp gồm kiểm tra domain/constraint, tích hợp PostgreSQL/Kafka thật, hợp đồng API, frontend DOM/build, live HTTP và kịch bản phục hồi. Mỗi lớp trả lời một câu hỏi khác nhau. Mock response phù hợp cho tương tác UI riêng lẻ, nhưng không dùng làm bằng chứng cho cạnh tranh khóa hoặc giao dịch PostgreSQL. Kiểm tra live UI dùng jsdom và fetch thật vẫn không thay visual QA trên Chrome.

| Thành phần máy đo T9.2 | Giá trị ghi trong E2.machine |
|---|---|
| Hệ điều hành và CPU | Windows 11 Home Single Language; i5-11400H, 2,70 GHz |
| Số nhân và luồng logic | 6 nhân,12 logical processor |
| RAM vật lý | 16.948.453.376 byte |
| Tài nguyên Docker | 12CPU, 8.208.916.480 byte bộnhớ được báo cáo |
| JVM backend và client tải | Xms256m/Xmx512m; Xms128m/Xmx256m |
| PostgreSQL và Kafka | 17.11 và 4.2.2, mỗi loại một node |
| Pool thực đo | Hikari max10/min2; chờ connection2000ms |

Ứng dụng, client tải và container dùng cùng máy, giao tiếp loopbackHTTP/1.1. Có warm cache và warm-up riêng, không có cam kết máy lạnh hoàn toàn. Cấu hình lock1500ms, statement/attempt5000ms, total10000ms, hold600 giây, expiry5 giây được lưu cùng số đo. Các giá trị máy là lịch sử phép đo, không suy từ máy hiện có vào ngày đọc báo cáo.

Các script tạo database/project/volume/cổng riêng. Test có thể reset fixture trong database UUID do chính harness sở hữu; không reset database demo người dùng. Khi nguồn chưa commit, gate phải overlay đầy đủ working tree và kiểm hash, không chứng nhận bản sửa bằng một clone HEAD cũ. T9.4 đã rà điểm này; T10.2 cũng đối chiếu 144 file runtime/harness giữa bản sao và working tree. [E4.source; E6.source]

T9.1 và T9.3 là các lượt kiểm chứng đã lưu, không phải lượt chạy lại khi viết báo cáo. Công việc T10.3 chủ yếu đọc, đối chiếu, xây tài liệu và kiểm tra bản xuất. Việc không chạy lại môi trường nghiệp vụ trong T10.3 giúp giữ dữ liệu demo và không tạo kết quả mới ngoài phạm vi yêu cầu.

## 6.2 Thiết kế phép đo 50 request đồng thời

T9.2 dùng Java platform thread với barrier: đủ 50 worker sẵn sàng rồi nhả cùng gate. Mỗi request có key riêng dưới một principal khách seed; đây là 50 request đồng thời, không phải 50 account khác nhau. Mỗi mode có 10 request warm-up, sau đó ba lượt 50 request trên fixture mới. Dữ liệu warm-up không gộp vào phân vị báo cáo. [E2.configuration, method; S1: tools/load/HttpBurst.java]

GA mỗi request quantity 1, capacity 10. SEATED mọi request cùng chọn cặp ghế có capacity 2, do đó chỉ một booking giữ đủ hai ghế. Độ trễ đo bằng System.nanoTime ngay trước send đến khi đọc hết response body. Không tự retry; tất cả response 202 hợp lệ,409 INVENTORY_UNAVAILABLE và lỗi khác được phân loại riêng.

```
Phân vị nearest rank: giá trị thứ ceil(p × n) sau sắp tăng
Throughput request/s: n response hoàn tất / thời gian burst
Booking thành công/s: số202 hợp lệ / thời gian burst
Với n=50, p99 bằng mẫu lớn nhất của lượt
```

Throughput toàn mode bằng tổng request chia tổng thời gian burst, không phải trung bình số học các throughput và không tính cả thời gian chuẩn bị. Phân vị toàn mode tính lại từ 150 mẫu, không lấy trung bình p95 của ba lượt. Phân vị trong bảng gồm cả conflict; số liệu cho response thành công được lưu riêng ở outcomes để tránh đánh đồng hai quần thể.

Sau burst, SQL đối chiếu các bookingId trả về, key, item, hold/history/outbox và allocation. Sau đó các request thắng mới được thanh toán mô phỏng, chờ CONFIRMED và truy vấn sold/tickets; phần này nằm ngoài booking latency. Nhờ hai snapshot, phép đo không chỉ chứng minh hold mà còn kiểm phân bổ sau bán.

Mỗi lượt ghi peakInFlight 50 và 50 distinct worker; số connection DB vẫn tối đa10. Gate chỉ đạt khi không có lỗi ngoài dự kiến và mọi counter SQL đúng. Response503,400, transport error hoặc202sai shape không được gọi là conflict hết vé. Các lỗi chuẩn bị/postprocess từng gặp được giữ trong lịch sử; bộ dữ liệu cuối được chọn vì quy trình kết thúc đầy đủ, không vì có latency thấp nhất. [E2.method, measurements, limits]

# CHƯƠNG 7 KẾT QUẢ THỰC NGHIỆM

## 7.1 Các gate và cách đọc số test

| Gate hoặc lượt | Backend thực chạy | Skip | Frontend | Contract |
|---|---|---|---|---|
| T9.1 | 130 | 0 | 54 | 69 |
| T9.3 | 130 | 0 | 54 | 69 |
| T9.4 local | 130 | 0 | 54 | 69 |
| T10.2 launcher | 9 | 121 | 54 | 69 |

Nguồn Bảng1: E1.results, E3.backend/frontend/contract/liveHttp, E4.results, E5.startupBuild, E6.checks. Các hàng là các lượt chạy khác nhau của nhiều test giống nhau; không cộng thành tổng số test độc lập của hệ thống.

Trong 130 test backend,121 ca cần PostgreSQL hoặc Kafka,9 ca còn lại là unit/configuration/H2 theo phân loại của gate. Khi chỉ chạy launcher không cấp biến IT, Maven vẫn có thể in Tests run130 nhưng Skipped121; chỉ9 ca thực thi. Vì vậy dùng dòng tổng Maven mà bỏ cột skipped sẽ dẫn đến kết luận sai về mức kiểm chứng của demo T10.2.

Ba lượt 74 response HTTP là222lần kiểm response, không phải222kịch bản khác nhau. Tương tự54testfrontend và69payloadcontract không tự là54luồngChrome hoặc69giao dịch toàn vẹn. Báo cáo giữ đơn vị đếm và loại kiểm tra để người đánh giá có thể đối chiếu với artifact gốc.

CI run39 được lưu success cho đúng commit984e95d và cả backend/frontend/contracts/secrets trong E4b. Bản auditT9.4 ban đầu còn ghi candidateGitHubNOT_RUN; đó là mốc trước khi bản sửa được push. Ghi nhận CI39 mới hơn giải quyết trạng thái này. Không dùng CI39 để chứng minh các file tài liệu/harness chưa commit ở T10.2/T10.3 đã có một GitHub Actions run mới.

Lịch sử T3/T4 trong PROGRESS cho thấy kiểm tra concurrency đã được đưa vào lúc triển khai GA và ghế:50 request choGA và các ca tậpghế/worker. Các mốc tuần là kế hoạch phụ thuộc kỹ thuật, không có căn cứ nói lịch thực tế đã kéo dài đúng mười tuần. T9.1 chạy lại các ca này; T9.2 bổ sung phương pháp đo có môi trường và mẫu chi tiết.

## 7.2 Số liệu tải theo từng lượt

| Lượt mode | 202 | 409 | Khác | p50 ms | p95 ms | p99 ms | Req/s |
|---|---|---|---|---|---|---|---|
| 1 GA | 10 | 40 | 0 | 213,476 | 259,192 | 261,075 | 189,172 |
| 1 SEATED | 1 | 49 | 0 | 102,921 | 149,046 | 155,954 | 319,725 |
| 2 GA | 10 | 40 | 0 | 136,478 | 172,829 | 176,717 | 276,512 |
| 2 SEATED | 1 | 49 | 0 | 92,563 | 137,852 | 144,967 | 344,825 |
| 3 GA | 10 | 40 | 0 | 162,774 | 198,168 | 206,285 | 238,570 |
| 3 SEATED | 1 | 49 | 0 | 98,667 | 131,083 | 134,074 | 358,332 |

Nguồn Bảng2: E2.measurements[0..5].summary và concurrency; độ trễms, throughput request/s; làm tròn ba chữ số thập phân. Mỗi hàng50 request và peakInFlight 50, không có lỗi khác. 409 trong bảng chỉ là INVENTORY_UNAVAILABLE đã phân loại đúng.

| Mode | Mẫu | 202 | 409 | Khác | p50 ms | p95 ms | p99 ms |
|---|---|---|---|---|---|---|---|
| GA | 150 | 30 | 120 | 0 | 163,883 | 251,749 | 260,020 |
| SEATED | 150 | 3 | 147 | 0 | 98,144 | 143,462 | 152,927 |

Nguồn Bảng3: E2.aggregates.GA/SEATED. Các giá trị tổng hợp tính trên 150 mẫu mỗi mode và tổng thời gian burst; không phải trung bình các phân vị ở Bảng2. CSV gốc E2a gồm mẫu warm-up và đo; phải lọc warmup=False khi tái tính.

GA có 30 booking được giữ trên ba fixture capacity 10; 120 request còn lại conflict đúng dự kiến. Với ghế, ba booking giữ tổng sáu ghế trên ba fixture cặp ghế, 147 request conflict. Con số success là booking, không phải số vé hoặc số tiền đã thanh toán.

Throughput request tương đối cao khi phần lớn request bị từ chối sớm không có nghĩa khả năng bán vé tương ứng. Giá trị booking thành công/s toàn mode là 45,821 cho GA và 6,804 cho SEATED, được tính trong cửa sổ burst; không bao gồm payment, phát vé và thời gian giữa lượt. [E2.aggregates.*.successfulBookingsPerSecond]

Chỉ có ba lượt và inventory nhỏ; chưa có khoảng tin cậy, phân phối người dùng độc lập, đường truyền TLS hoặc tải duy trì. p99 của50 mẫu bằng max nên nhạy với từng mẫu; không nên dùng các số này làm SLO production. Các mẫu và cách tính vẫn hữu ích để lặp lại cùng kịch bản trên cấu hình đã công bố.

## 7.3 Đối chiếu PostgreSQL sau hold và sau bán

| Mode | Mốc | Cap | Held | Sold | Booking | Vé | Oversell |
|---|---|---|---|---|---|---|---|
| GA | Sau giữ | 10 | 10 | 0 | 10 | 0 | 0 |
| GA | Sau bán | 10 | 0 | 10 | 10 | 10 | 0 |
| SEATED | Sau giữ | 2 | 2 | 0 | 1 | 0 | 0 |
| SEATED | Sau bán | 2 | 0 | 2 | 1 | 2 | 0 |

Nguồn Bảng4: E2.measurements[*].afterHold/afterSale; các lượt cùng mode có cùng counter allocation nhưng ID fixture khác nhau. Không phải truy vấn lấy số seed100 để chứng minh capacity 10.

Trong GA, mười response 202 khớp mười bookingId, mười key, mười item quantity 1 và mười BookingHeld. Trước payment: held10/sold0/0 vé. Sau payment các request thắng: held0/sold10/10 vé/10CONFIRMED. Số allocation và các ID phải tương ứng với response; chỉ đếm tổng booking toàn DB sẽ không phát hiện tạo nhầm ngoài fixture.

Trong SEATED, một request thắng giữ hai ghế, có một booking/key và hai item. Sau bán hai ghế SOLD cùng owner hợp lệ và hai vé gắn đúng item; không có ticket trùng ghế hoặc ordinal. Các request thua không để lại booking/item/key chưa hoàn chỉnh hoặc ghế bên thua bị giữ một phần.

Tất cả sáu lượt có oversell0, tổng counter vi phạm0 và rollbackArtifacts0 theo định nghĩa harness. rollbackArtifacts là kết luận tổng hợp từ các counter như partialBookings, incompleteKeys, unmappedBookings, owner mismatch và unexpectedKeys; không phải một metric độc lập được đo bởi hệ quản trị.

Các CHECK/FK/unique là lớp bảo vệ, nhưng bằng chứng còn cần các truy vấn đối chiếu quantity và trạng thái. T9.2 lưu SQL đầy đủ ở E2b, CSV ở E2a và snapshot JSON trong E2. Database thử có thể được giữ lại hoặc fixture được tạo mới khi chạy gate; không mặc định mọi UUID trong báo cáo vẫn tồn tại trên môi trường đang mở.

Không có khẳng định rằng sáu burst đã bao phủ mọi kiểu rollback, deadlock hoặc mất kết nối tại COMMIT. Những ca lỗi có kiểm soát của T9.1/T9.3 được trình bày riêng để không suy quá mức từ kết quả tải không oversell.

## 7.4 Kafka outage duplicate retry DLQ và replay

| Ca T9.3 | Trước hoặc trong lỗi | Sau phục hồi |
|---|---|---|
| Consumer restart/duplicate | 3 vé | 3 vé; historyCONFIRMED=1 |
| Ba delivery lỗi | Inbox0, vé0 | Inbox1, vé2, RESOLVED1 |
| Poison hai consumer | Mỗi consumer3attempt | 2failure và2DLQ |
| Replay payment/confirmed | 2vé/2history | Hash trước=sau; không thêm nghiệp vụ |

Nguồn Bảng5: E3.comparisonProofs. Mỗi hàng thuộc fixture riêng; số vé2 và3 không phải các mốc của cùng một booking. Bằng chứng chứa tên criterion và counter trước/sau để tra cứu chính xác.

Khi broker dừng, ledger đã commit không đổi, outbox giữ việc cần gửi; sau phục hồi có ba event publish và hai vé của booking hợp lệ. Consumer restart cùng duplicate event không tạo vé thứ tư hoặc confirm history thứ hai. Cửa sổ ACK/DB mark lỗi được tiêm bằng trigger để chứng minh resend cùng eventId, không phải khẳng định đã kill tiến trình đúng mọi điểm máy lệnh.

Consumer lỗi ba delivery rollback inbox và phát vé, giữ record failure trước ACK rồi gửi DLQ. Sau sửa nguyên nhân và replay có kiểm soát, có đúng receipt và đúng vé. Với poison record, mỗi consumer giữ failure và phát DLQ riêng. Các failure cố ý này là kết quả mong đợi; không phải lỗi hệ thống bị giấu bằng nhãn PASS.

Projection gap cần khôi phục theo version; payment consumer không dùng version hiện tại của booking để loại bỏ kết quả. Quyền ADMIN,401/403, event không tồn tại và budget replay được kiểm trong suite. Một request mới vượt budget bị409 còn request cũ trả cached response; không có thao tác xóa inbox hoặc force replay trong ca đạt.

Hash nghiệp vụ bằng nhau trước/sau HTTP replay cùng với receipt DUPLICATE là bằng chứng no-op mạnh hơn chỉ thấy202. Audit publisher có thể đổi một cách hợp lệ, nhưng payment/item/ticket/inventory/history phải giữ nguyên. Các cửa sổ ngoài phạm vi fault injection được giữ trong phần giới hạn, không gọi là phục hồi tuyệt đối.

## 7.5 Payment expiry rollback và khởi động lại

T9.3 lưu 37 snapshot của 13 nhóm kịch bản, mỗi snapshot kiểm 11 counter vi phạm, tất cả bằng 0. Có 407 lần kiểm counter, không phải 407 test case độc lập. Các counter gồm oversell, GA quantity mismatch, hold mồ côi, ghế active trùng/owner sai, vé không hợp lệ/trùng ghế, history/event nghiệp vụ trùng, compensation trùng và completion trùng. [E3.snapshots; E4.results.sumViolations]

| Tình huống | Kết quả thực trong E3.comparisonProofs |
|---|---|
| Payment đúng hạn, Kafkađãtắt | Worker xử lý2 vé, inbox payment0; sau phục hồi 1 receipt, không EXPIRED |
| SuccesssauEXPIRED và refund retry | REFUND_REQUIRED,0 vé,1nghĩa vụ,2 attempt,1completion |
| Ghế đã cấp chủ mới | Booking cũ0 vé, REFUND_REQUIRED; snapshot 2 ghế HELD của owner mới không đổi |
| Payment FAILURE lặp | GA/ghế PAYMENT_FAILED,0 vé; history/inventory không đổi |
| Delayed result bền | Ledger SCHEDULED; booking cuối REFUND_REQUIRED; 1 result event, 1 compensation, 0 vé |

Trong ca broker thật dừng, production expiry service được gọi có kiểm soát, scheduler trong test Kafka tắt để định thời xác định. Việc scheduler thật dừng/resume và JVM restart được thử riêng trong live script. Không gộp hai loại ca này thành tuyên bố đã thử mọi lịch xen kẽ của thread.

Fault injection PostgreSQL tại ghi outbox, vé, history hoặc compensation kiểm rollback nhất quán. Nếu record result đã commit nhưng xử lý ticket lỗi, result vẫn bền để thử lại; không gọi cả hai transaction là một atomic transaction. Restart JVM và payment SCHEDULED kiểm cùng database/intent, migration không chạy trùng và late payment không phát vé. [E1.coverageReview; E3.sqlAndLiveChecks]

CI38 từng thất bại do test đọc log HTTP trước khi finally của CorrelationFilter hoàn tất, dù client đã nhận body. Bản sửa đồng bộ đúng mốc log và test hồi quy giữ assertion; không sửa nghiệp vụ hoặc đổ cho Node warning. Backend 130/130 đạt ở môi trường tương đương CI, sau đó CI39 đúng SHA xanh. Trường hợp này minh họa cần phân biệt race của kiểm thử với race dữ liệu thật. [E4c; E4b]

## 7.6 Demo sạch và lượt tập 5 đến 10 phút

T10.1 dựng clone, project, database, volume và cổng riêng theo README, giữ dữ liệu demo cũ. Chủ dự án xác nhận Chrome: booking GA CONFIRMED có một vé, booking A1 CONFIRMED có vé đúng ghế, history và dashboard admin hiển thị đúng hai booking. Agent không truy cập được file ảnh nên không có ảnh Chrome đưa vào báo cáo. HTTP/SQL sau Stop/Start giữ 18 bảng và 9 row Flyway, hai booking/hai vé. [E5; E5a]

| Hạng mục T10.2 | Kết quả ghi nhận |
|---|---|
| Lượt kỹ thuật | 8/8 bước; 52,298 giây |
| Lượt có nhịp trình diễn | 8/8 bước; 365,499 giây |
| Khoảng xem/thuyết minh | 310 giây được đặt |
| Lệnh và overhead còn lại | 55,499 giây |
| Stop/Start | Exit0, healthUP;18hashbảng/9migration giữ nguyên |
| Chrome trong T10.2 | Chưa thao tác hoặc quan sát bởi agent |

Nguồn Bảng6: E6.main.timing, repeatability, checks và restart. Khoảng xem/thuyết minh được đặt là 310 giây; phần còn lại gồm xử lý, lưu kết quả và overhead, không phải CPU time. Backup và setup nằm ngoài đồng hồ chính.

Lượt chính T10.2 sử dụng hai fixture mới: GA 10 vé và SEATED A1/A2, TTL 20 giây ở process để không đợi 600 giây. Kết thúc có 4 booking: hai CONFIRMED, một REFUND_REQUIRED, một CANCELLED; 2 vé, 9 history, 12 outbox, 15 inbox và một compensation COMPLETED. GA held0/sold1/available9, A1 SOLD/A2 AVAILABLE; mười counter cuối đều 0. Không lấy tổng DB gồm các lượt trước để làm assertion.

Một lượt kỹ thuật trước đạt 8/8 trong 52,298 giây, không được gọi là buổi trình diễn 5–10 phút. Phần dự phòng trên fixture ghế khác kiểm EXPIRED→đặt lại→FAILURE→AVAILABLE, không có vé; đổi payload cùng key bị409. Hai .env có sẵn và bốn evidence chưa tracked ban đầu giữ nguyên; dừng bằng Stop giữ volume.

T10.2 chưa thao tác hoặc quan sát Chrome bởi agent; chỉ có chuỗi lệnh HTTP/SQL và khoảng dừng định giờ. Chín test launcher pass/121 integration skip không thay 130 test đã chạy ở gate trước. Sự phân biệt này giúp buổi bảo vệ không gán tính đúng đắn của giả lập hoặc ảnh người dùng cho một loại kiểm chứng khác.

# CHƯƠNG 8 ĐÁNH GIÁ VÀ KẾT LUẬN

## 8.1 Kết quả đạt được và hướng phát triển

Sản phẩm đã có luồng GA và ghế từ catalog đến booking, payment mô phỏng, vé và lịch sử; admin quản lý catalog và xem khôi phục. PostgreSQL giữ quyền phân bổ, idempotency và lịch sử trong transaction; outbox/inbox bổ sung đường khôi phục khi broker hoặc consumer lỗi. Các gate chứng minh được những bất biến cụ thể trên fixture và môi trường đã công bố.

Giá trị kỹ thuật của đồ án là làm rõ ranh giới bền vững và hiệu ứng lặp. Availability không cấp quyền giữ chỗ; countdown không tự nhả; SUCCESS muộn không chiếm lại vé; HTTP202 hoặc Kafka ACK không chứng minh mọi nghiệp vụ đã xong. Các nguyên tắc này được hiện thực bằng dữ liệu, transaction, guard và kiểm thử với lỗi có kiểm soát.

| Giới hạn hiện tại | Hướng kiểm chứng hoặc nâng cấp sau MVP |
|---|---|
| BrokerRF1, một máy | TestHA, mất broker/ổđĩa, networkpartition và restore backup |
| Burst ngắn, ít lượt | Tải duy trì, SLO được thống nhất, profiling và khoảng tin cậy |
| Auth/paymentsimulation | Tích hợp identity/providerthật bằng ADR và đối soát settlement |
| Chưa thử mọi crash window | Fault campaign ởCOMMIT/offset, pool exhaustion độc lập |
| UI giới hạn100recordcustomer | Thiết kế phân trang và kiểm tra trực tiếp Chrome/mobile/accessibility |
| Event/key lưu dài | Chính sách retention, archive, version và quyền truy cập audit |

Không có bằng chứng cho tiền thật, exactly-once xuyên hệ, production QR, quy mô Internet hoặc mọi lịch sử đồng thời. Scanner và các log đã thu không chứng minh không thể lộ bí mật ở mọi cấu hình. Không có ảnh giả để lấp phần Chrome chưa được kiểm; việc hoàn thiện giao diện trực quan cần được đánh giá tiếp khi có điều kiện.

Các mục nâng cấp là đề xuất, không phải tính năng đã làm trong T10.3. Bước đóng gói/tag và chốt backlog thuộc T10.4 chưa triển khai. Báo cáo kết luận trong phạm vi đã đo: các bất biến quan trọng được giữ ở các gate và kịch bản đã chạy, đồng thời cung cấp lệnh và nguồn để người đọc lặp lại thay vì chỉ tin vào nhận định tổng quát.

# TÀI LIỆU THAM KHẢO

Nguồn kỹ thuật chính thức được truy cập ngày 14/11/2026; các phần giải thích triển khai lấy từ source và ADR, không sao chép nguyên văn tài liệu ngoài.

[R1] PostgreSQL Global Development Group. PostgreSQL17 Documentation, Transaction Isolation. https://www.postgresql.org/docs/17/transaction-iso.html

[R2] PostgreSQL Global Development Group. PostgreSQL17 Documentation, Explicit Locking. https://www.postgresql.org/docs/17/explicit-locking.html

[R3] Apache Software Foundation. Apache Kafka 4.2 Design. https://kafka.apache.org/42/design/design/

[R4] Spring. Spring for Apache Kafka, Handling Exceptions. https://docs.spring.io/spring-kafka/reference/kafka/annotation-error-handling.html

[R5] Spring. Spring Boot, Logging. https://docs.spring.io/spring-boot/reference/features/logging.html

[R6] OpenAPI Initiative. OpenAPI Specification 3.1.0, 2021. https://spec.openapis.org/oas/v3.1.0.html

[R7] Redgate. Flyway Migrations. https://documentation.red-gate.com/fd/migrations-271585107.html

[D1] Lê Hữu Hào. PRD, ARCHITECTURE, DESIGN, DATA_MODEL, RULES, TEST_PLAN, PROGRESS; ADR0001–0008 trong docs/ của repository event-ticketing-platform, mốc đối chiếu14/11/2026.

[S1] Repository event-ticketing-platform, source backend/frontend, migrationV1–V9, OpenAPI vàJSONSchema; HEAD984e95d cùng working tree mô tả trong mụcThuậtngữ. https://github.com/Haolo333z/event-ticketing-platform

[E1] docs/evidence/t9-1-invariants-2026-11-14.json. Gate bất biến, nguồn aa8b9f3, phân loại130 test và các giới hạn.

[E2] docs/evidence/t9-2-load-2026-11-14.json; [E2a] t9-2-samples-2026-11-14.csv; [E2b] t9-2-audits-2026-11-14.sql, cùng thư mục. Cấu hình, mẫu vàSQL sáu burst đo.

[E3] docs/evidence/t9-3-recovery-2026-11-14.json; t9-3-snapshots-2026-11-14.json và t9-3-queries-2026-11-14.sql.37 snapshot/13 nhóm và comparisonProofs.

[E4] docs/evidence/t9-4-audit-2026-11-14.json; [E4b] t9-4-ci39-2026-11-14.json; [E4c] ci38-backend-fix-2026-11-14.json, cùng thư mục. CI39: https://github.com/Haolo333z/event-ticketing-platform/actions/runs/43563069678

[E5] docs/evidence/t10-1-clean-demo-2026-11-14.json; [E5a] t10-1-chrome-user-2026-11-14.md. Nguồn Chrome là xác nhận người dùng, không ảnh đã xem.

[E6] docs/evidence/t10-2-rehearsal-2026-11-14.json và docs/DEMO_AND_RECOVERY.md. Timings, fixture, main/backup, no-op và bảo toàn môi trường.

# PHỤ LỤC A LỆNH TÁI HIỆN TRÊN WINDOWS

Chạy từ repository, Docker Desktop Linux engine đã sẵn sàng; Java/Node theo README. Không sử dụng lệnh xóa volume hoặc reset DB. Gate có thể tạo tài nguyên kiểm thử riêng; đọc hướng dẫn từng script và giữ .env hiện có. Với working tree chưa commit phải dùng gate có overlay đúng nguồn.

```
cd D:\event-ticketing-platform
git status --short
git rev-parse HEAD
docker version
powershell -NoProfile -ExecutionPolicy Bypass `
  -File infra/test-invariants.ps1 -WorkingTree
powershell -NoProfile -ExecutionPolicy Bypass `
  -File infra/test-load.ps1 -Requests 50 -Rounds 3
npm.cmd test --prefix tools/contracts
npm.cmd test --prefix frontend
npm.cmd run build --prefix frontend
```

Gate WorkingTree ở mốc hiện hành tái sử dụng kiểm tra phục hồi và SQL; không chạy verify-part7 trên DB demo cần bảo toàn vì script đó ghi fixture. Các gate T9 đã chạy được ghi trong E1–E4; các lệnh trên là hướng dẫn tái hiện, không phải tuyên bố đã chạy lại trong T10.3.

```
# Môi trường T10.2 đã Prepare: Start nếu đã dừng
powershell -NoProfile -ExecutionPolicy Bypass `
  -File infra/demo-rehearsal.ps1 -Action Start
powershell -NoProfile -ExecutionPolicy Bypass `
  -File infra/demo-rehearsal.ps1 -Action NewRun
powershell -NoProfile -ExecutionPolicy Bypass `
  -File infra/demo-rehearsal.ps1 -Action Rehearse -Paced
# Dừng đúng session, giữ volume
powershell -NoProfile -ExecutionPolicy Bypass `
  -File infra/demo-rehearsal.ps1 -Action Stop
```

Lần đầu dùng Prepare theo DEMO_AND_RECOVERY; nếu cổng bận script phải từ chối, không kill dịch vụ khác. UI riêng http://127.0.0.1:5173, API18081; T10.1 dùng localhost:5173 qua IPv6 và API18080. Dùng đúng URL, persona và fixture NewRun in ra, không nhầm hai môi trường. Không đưa token/password hoặc raw payment vào output bảo vệ.

# PHỤ LỤC B TRUY VẤN ĐỐI CHIẾU CHỈ ĐỌC

Truy vấn phải lọc đúng fixture, không lấy tổng seed làm assertion. SQL dưới minh họa dùng psql variable ga_event; chạy trên DB test do chính mình tạo và kiểm project trước. Kết quả0 row nghĩa là không tìm thấy lỗi trong scope, không phải chứng minh mọi dữ liệu ở mọi thời điểm.

```
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ READ ONLY;
SELECT g.ticket_type_id, g.capacity, g.held_count, g.sold_count
FROM ticketing.inventory_ga g
JOIN ticketing.ticket_types t ON t.id=g.ticket_type_id
WHERE t.event_id=:'ga_event'::uuid
  AND (g.held_count<0 OR g.sold_count<0
       OR g.held_count+g.sold_count>g.capacity);

SELECT i.seat_id, count(*) AS active_allocations
FROM ticketing.booking_items i
JOIN ticketing.bookings b ON b.id=i.booking_id
WHERE b.event_id=:'seat_event'::uuid
  AND i.seat_id IS NOT NULL
  AND b.status IN ('PENDING_PAYMENT','CONFIRMED')
GROUP BY i.seat_id HAVING count(*)>1;

SELECT b.id, b.status, count(t.id) AS unexpected_tickets
FROM ticketing.bookings b
JOIN ticketing.tickets t ON t.booking_id=b.id
WHERE b.event_id IN (:'ga_event'::uuid,:'seat_event'::uuid)
  AND b.status<>'CONFIRMED'
GROUP BY b.id,b.status;
COMMIT;
```

Để kiểm quantity, owner, key hoàn chỉnh, history trùng, compensation và projection, dùng full query infra/demo-audit.sql, E2b và E3 queries. Các đoạn trên không thay đầy đủ bộ counter. Trước/sau replay so sánh hash đã loại publisher audit nhưng giữ ledger, vé, inventory và receipt để kiểm no-op.

```
$s=Get-Content reports/t10-2/session.json -Raw|ConvertFrom-Json
$r=Get-Content $s.runFile -Raw|ConvertFrom-Json
Get-Content infra/demo-audit.sql -Raw | docker exec -i `
  "$($s.project)-postgres-1" psql -U ticketing `
  -d ticketing_rehearsal -qAt -v ON_ERROR_STOP=1 `
  -v "ga_event=$($r.ga.id)" -v "seat_event=$($r.seated.id)"
```

# PHỤ LỤC C MA TRẬN TRUY VẾT VÀ BÀN GIAO

| Nhận định | Source hoặc ADR | Bằng chứng thực |
|---|---|---|
| GA không oversell | GaInventory, ADR0004 | E1 GA-01; E2 afterHold/afterSale |
| Ghế nguyên tử, owner đúng | SeatInventory, V7, ADR0005 | E1 SEAT-01/02; E2 SEATED |
| Key trùng không booking mới | BookingCommandService,V5 | E1 IDEM; E6 Ga |
| Expiry chủ động, release một lần | ExpiryService, BookingLifecycle | E1 EXP-01; E3 snapshots |
| Payment đúng hạn/late | PaymentResultProcessor,ADR0006 | E3 comparisonProofs; E6 Late |
| Vé đúng số và không phát lặp | TicketService,V8 | E1 TICKET; E3/E6 Replay |
| Outbox sau commit và resend | OutboxWriter/Publisher,ADR0007 | E3 ACK/DBmark/outage |
| Inbox/rollback/retry/ DLQ | EventConsumer,FailureStore,V9 | E3 comparisonProofs |
| Replay có quyền/budget/audit | ReplayService, RecoveryQuery | E3 ADMIN; E6 Replay |
| Schema/seed/restart giữ nguyên | V1–V9, local seed | E4 migration; E5/E6 restart |
| Log/health/metrics | operations, ARCHITECTURE T8 | E1/E3; E6 outage |
| Chrome GA/A1/history/admin | Nguồn xác nhận người dùng | E5a, không có ảnh đã xem |

Các bảng định lượng của báo cáo lấy trực tiếp từ JSON evidence khi build; kiểm tra độc lập CSV lọc warm-up, nearest-rank và throughput giúp bắt lỗi chép hoặc gộp phân vị. Mục lục dùng heading/bookmark và page number được đối chiếu với bản PDF cuối; sơ đồ có nguồn và không được gọi là screenshot.

Bản tại ngày đối chiếu giữ các file khác của T10.1–T10.2 chưa commit. Thông tin trường, khoa, MSSV, lớp, giảng viên và ngày nộp trên bìa cần người nộp điền; không có suy đoán học viện hay chức danh. Tên sinh viên lấy từ PRD và kế hoạch cũ. Khi có mẫu trường, cần áp dụng mẫu và kiểm tra lại pagination trước nộp.

Các giới hạn kỹ thuật và phạm vi đã đánh giá nằm ở Chương8. Hoàn thành báo cáo không có nghĩa hoàn thành T10.4 hoặc một bản phát hành production. Bằng chứng kiểm tra xuất bản, số trang và layout của T10.3 được ghi riêng trong PROGRESS và evidence T10.3, không cộng vào số test nghiệp vụ.